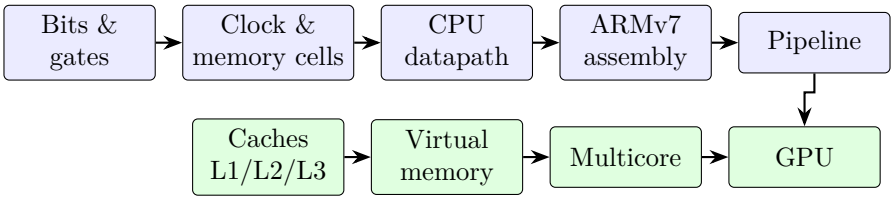
Really Knowing the Computer

From Transistors to GPUs

A Visual Course in Computer Architecture

How to use this course

This course takes you from a single transistor all the way to a modern CPU with caches, virtual memory and a GPU. The path is deliberately bottom-up:



Conventions.

- **Yellow boxes** are worked examples: follow them with pen and paper, step by step.
- **Blue boxes** are the ideas to remember if you remember nothing else.
- **Red boxes** are classic traps.
- Binary numbers are written `1011`, hexadecimal as `0x2F`.
- Every chapter ends with exercises; short answers are in the appendix.
- The assembly language is **ARMv7-A** (32-bit ARM, the instruction set of Raspberry Pi 2, many phones of the 2010s, and the base of everything ARM does today).
- Hardware numbers (latencies, sizes) are typical orders of magnitude, not the specs of one product. Use them to build intuition.

Contents

1 The Big Picture and Data Representation **1**
- 1.1 What is a computer, really? . . . 1
  - 1.1.1 The layers of abstraction . . . 1
  - 1.1.2 The von Neumann model . . . 2
- 1.2 Bits, bytes and number systems . . . 2
  - 1.2.1 Binary and hexadecimal . . . 2
  - 1.2.2 Negative numbers: two's complement . . . 3
  - 1.2.3 Overflow . . . 4
  - 1.2.4 Endianness: byte order in memory . . . 4
  - 1.2.5 Characters and floating point . . . 4
- 1.3 Units you must know . . . 4

2 Transistors and Logic Gates **6**
- 2.1 The transistor: a switch controlled by voltage . . . 6
  - 2.1.1 The CMOS inverter (NOT gate) . . . 6
  - 2.1.2 NAND and NOR from four transistors . . . 7
- 2.2 Logic gates and truth tables . . . 7
- 2.3 Boolean algebra . . . 7
- 2.4 Combinational building blocks . . . 8
  - 2.4.1 Multiplexer (mux): the "if" of hardware . . . 8
  - 2.4.2 Decoder . . . 8
  - 2.4.3 Adders . . . 9
  - 2.4.4 The ALU (Arithmetic Logic Unit) . . . 9

3 Memory Cells and the Clock **11**
- 3.1 From gates to memory: feedback . . . 11
  - 3.1.1 D latch and D flip-flop . . . 11
- 3.2 The clock . . . 12
  - 3.2.1 The timing rule: setup, hold and the critical path . . . 12
  - 3.2.2 Why clock speed stopped growing . . . 13
- 3.3 Finite state machines: a counter . . . 13
- 3.4 Registers, register files and the first memory . . . 14

4 The CPU: Datapath and Control **15**
- 4.1 The five ingredients . . . 15
- 4.2 What does an instruction look like? . . . 15
- 4.3 Fetch, decode, execute: one instruction in detail . . . 16
- 4.4 Control signals . . . 17
- 4.5 RISC versus CISC . . . 17
- 4.6 Why the single-cycle CPU is too slow . . . 17

5 ARMv7 Assembly Language **19**
5.1 The programmer's view of an ARM core . . . 19
5.1.1 Registers . . . 19
5.1.2 The status register CPSR and its flags . . . 19
5.2 Data-processing instructions . . . 20
5.2.1 The barrel shifter: a free shift on operand 2 . . . 20
5.2.2 Immediates: the 8-bit rotated constant rule . . . 20
5.3 Comparisons and conditional execution . . . 21
5.4 Branches and loops . . . 21
5.5 Memory access: load and store . . . 22
5.5.1 Arrays in memory . . . 22
5.5.2 Multiple registers and the stack . . . 23
5.6 Functions and the calling convention . . . 23
5.7 A complete program: Hello, world on Linux . . . 24
5.8 What the compiler really does . . . 25
5.9 Thumb, NEON and beyond . . . 25

6 Performance and Pipelining **26**
6.1 How do we measure speed? . . . 26
6.1.1 Amdahl's law . . . 26
6.2 The pipeline idea . . . 27
6.3 Hazards: when the pipeline cannot flow . . . 27
6.3.1 Structural hazards . . . 27
6.3.2 Data hazards . . . 28
6.3.3 Control hazards: branches . . . 28
6.4 Beyond the simple pipeline . . . 29

7 The Memory Hierarchy: Registers, L1, L2, L3, RAM **30**
7.1 The memory wall . . . 30
7.2 Why the hierarchy works: locality . . . 30
7.3 Memory technologies . . . 31
7.4 How a cache works . . . 31
7.4.1 Splitting an address . . . 31
7.4.2 Direct-mapped cache . . . 32
7.4.3 Conflicts and associativity . . . 32
7.4.4 Types of misses (the three C's) . . . 33
7.4.5 Writes . . . 33
7.5 Measuring cache performance: AMAT . . . 33
7.6 The real L1 / L2 / L3 hierarchy . . . 34
7.7 Writing cache-friendly code . . . 34

8 Virtual Memory **36**
8.1 The problem . . . 36
8.2 Pages and page tables . . . 36
8.3 Page-table size and multi-level tables . . . 37
8.4 The TLB: a cache for translations . . . 38
8.5 Page faults: using the disk as extra memory . . . 38
8.6 Memory protection and privilege . . . 39
8.7 Caches and virtual addresses . . . 39

9 Inside a Modern CPU: Parallelism at Every Level **40**
9.1 Superscalar and out-of-order execution . . . 40
9.2 SIMD: one instruction, many data . . . 41
9.3 Multicore and multithreading . . . 41
9.4 Cache coherence . . . 42
9.5 Memory ordering and synchronisation . . . 43

10 The GPU: Massive Parallelism **44**
10.1 Two design philosophies . . . 44
10.2 The SIMT execution model . . . 44
10.2.1 Your first kernel: vector addition . . . 45
10.3 Warp divergence . . . 45
10.4 GPU memory hierarchy . . . 46
10.4.1 Coalescing . . . 46
10.4.2 Hiding latency with many threads . . . 46
10.5 How fast, really? The roofline model . . . 46
10.6 Talking to the GPU . . . 47
10.7 The graphics pipeline in one picture . . . 47
10.8 CPU vs GPU: when to use which . . . 48

11 I/O, Interrupts and the Whole Machine **49**
11.1 Talking to devices . . . 49
11.1.1 Three ways to wait for a device . . . 49
11.2 Interrupts and exceptions . . . 49
11.3 Buses and bandwidth . . . 50
11.4 From power button to login prompt . . . 50
11.5 The grand tour: one load, from transistor to DRAM . . . 51
11.6 Numbers every engineer should know . . . 51
11.7 Where to go next . . . 51

A ARMv7 Cheat Sheet **53**

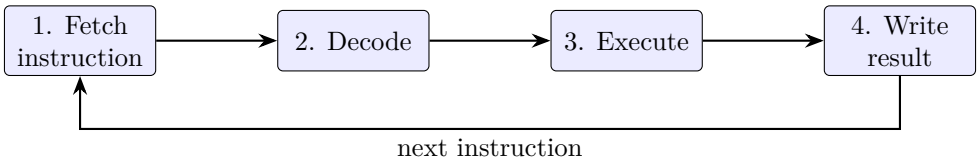
B Short Answers to Exercises **55**

Chapter 1

The Big Picture and Data Representation

1.1 What is a computer, really?

A computer is a machine that repeats one tiny loop billions of times per second:



Everything else (windows, games, neural networks) is built on top of this loop. This course explains *how* the loop is physically done, and then how engineers make it hundreds of times faster using clever tricks (pipelines, caches, parallelism).

1.1.1 The layers of abstraction

No one can think about billions of transistors at once. We split the computer into layers; each layer only needs to know the one just below it.

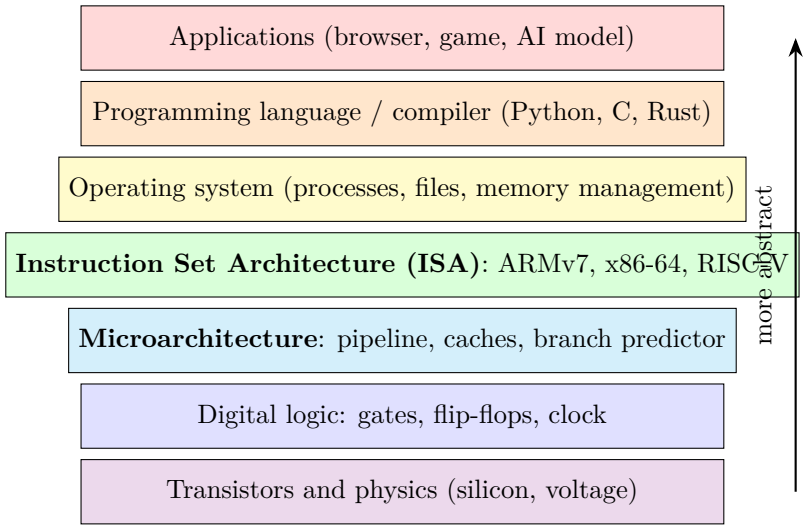


Figure 1.1: The stack of abstractions. This course lives in the green, cyan, and blue layers.

Key idea

The **ISA** (Instruction Set Architecture) is the *contract* between software and hardware: it says which instructions exist and what they do. The **microarchitecture** is *how* a particular chip fulfils that contract. Two chips can share one ISA (e.g. ARMv7) and have completely different internals and speed.

1.1.2 The von Neumann model

Almost every computer follows the same plan: a CPU, a memory that stores *both* program and data, and input/output devices, connected by buses.

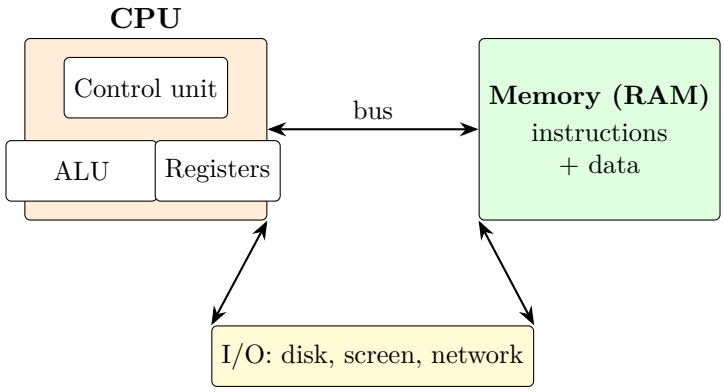


Figure 1.2: The von Neumann architecture.

Analogy

Think of a kitchen. The *recipe book* and *ingredients* are in the pantry (memory). The *cook* (CPU) can only work with what is on the counter (registers). The cook fetches things from the pantry, works on them, and puts results back. The recipe itself is stored in the same pantry: that is the "stored program" idea.

1.2 Bits, bytes and number systems

A **bit** is a value that is 0 or 1 (a low or high voltage). A **byte** is 8 bits. A **word** is the natural chunk of a CPU; on ARMv7, a word is 32 bits (4 bytes).

1.2.1 Binary and hexadecimal

In decimal each digit is worth a power of 10; in binary each digit is worth a power of 2.

bit position	7	6	5	4	3	2	1	0
weight	128	64	32	16	8	4	2	1

Worked example 1.1: Binary to decimal

Convert `10110101` to decimal.

1. Write the weights under the bits: `1`→128, `0`→64, `1`→32, `1`→16, `0`→8, `1`→4, `0`→2, `1`→1.
2. Keep only the weights where the bit is 1: $128 + 32 + 16 + 4 + 1$.
3. Add: 181.

Worked example 1.2: Decimal to binary (repeated division)

Convert 45 to binary.

1. $45 \div 2 = 22$ remainder **1**
2. $22 \div 2 = 11$ remainder **0**
3. $11 \div 2 = 5$ remainder **1**
4. $5 \div 2 = 2$ remainder **1**
5. $2 \div 2 = 1$ remainder **0**
6. $1 \div 2 = 0$ remainder **1**
7. Read remainders from bottom to top: `101101`. Check: $32 + 8 + 4 + 1 = 45$. ✓

Binary numbers are long, so we group bits by 4 and write each group as one **hexadecimal** digit (0–9, A–F, where A=10, …, F=15).

bin	dec	hex	bin	dec	hex	bin	dec	hex
0000	0	0	0110	6	6	1100	12	C
0001	1	1	0111	7	7	1101	13	D
0010	2	2	1000	8	8	1110	14	E
0011	3	3	1001	9	9	1111	15	F
0100	4	4	1010	10	A			
0101	5	5	1011	11	B			

Worked example 1.3: Binary ↔ hex

`1101 0110 0011 1010` → `D`, `6`, `3`, `A` = `0xD63A`. Going back: `0x2F` → `0010 1111`.

1.2.2 Negative numbers: two's complement

How do we store -5? With n bits, **two's complement** gives the top bit weight -2^{n-1} instead of $+2^{n-1}$. The range of n bits is -2^{n-1} to $2^{n-1} - 1$ (for 8 bits: -128 to 127; for 32 bits: about ± 2.1 billion).

Worked example 1.4: Negating a number

Find the 8-bit representation of -5.

1. Start from $+5$ = `00000101`.
2. **Invert** every bit: `11111010`.
3. **Add 1**: `11111011` = `0xFB`.
4. Check using weights: $-128 + 64 + 32 + 16 + 8 + 2 + 1 = -128 + 123 = -5$. ✓

Key idea

Two's complement is brilliant because **the same adder circuit works for signed and unsigned numbers**. Subtraction $a - b$ is just $a +$ (invert b) $+ 1$.

Worked example 1.5: Subtraction with an adder

Compute $7 - 5$ in 8 bits.

1. 7 = `00000111`, 5 = `00000101`.
2. Invert 5: `11111010`; add 1 ⇒ `11111011` ($= -5$).
3. Add: `00000111` + `11111011` = `1 00000010`.
4. Drop the carry-out (9th bit): `00000010` $= 2$. ✓

Careful!

The bits `11111011` mean 251 if you treat them as *unsigned* and −5 if *signed*. The hardware does not know which; *you* (the compiler, the programmer) decide. This is why ARM has both signed and unsigned branch conditions.

1.2.3 Overflow

If a result does not fit in the available bits, it "wraps around". In 8-bit signed: $127 + 1 = -128$. Hardware detects this with flags (Chapter 5).

1.2.4 Endianness: byte order in memory

A 32-bit word `0x12345678` occupies 4 memory bytes. **Little-endian** (ARM in practice, x86) puts the *least significant* byte at the *lowest* address.

address	`0x1000`	`0x1001`	`0x1002`	`0x1003`
little-endian	`0x78`	`0x56`	`0x34`	`0x12`
big-endian	`0x12`	`0x34`	`0x56`	`0x78`

1.2.5 Characters and floating point

Text is stored as numbers: ASCII `'A'`= 65 =`0x41`, `'a'`= 97 =`0x61`, `'0'`= 48 =`0x30`. Modern text uses UTF-8 (1 to 4 bytes per character, ASCII-compatible).

Real numbers use **IEEE 754** floating point. A 32-bit float has 1 sign bit, 8 exponent bits, 23 fraction bits:

$$(-1)^s \times 1.f \times 2^{e-127}.$$

Worked example 1.6: Encoding 0.15625 as a float

1. $0.15625 = 5/32 =$ `0.00101` in binary.
2. Normalise: 1.01×2^{-3}.
3. Sign $s = 0$. Exponent field $e = -3 + 127 = 124 =$ `01111100`.
4. Fraction: the digits after the leading 1 are `01`, padded with zeros to 23 bits.
5. Bits: `0 01111100 01000000000000000000000` = `0x3E200000`.

Careful!

0.1 has no exact binary form (it is `0.000110011...` forever), so in floating point $0.1+0.2 \neq 0.3$ exactly. This is a hardware reality, not a bug in your language.

1.3 Units you must know

Prefix	Power of 2 (memory)	Power of 10 (speed, disks)
Kilo	$2^{10} = 1024$ (KiB)	10^3
Mega	$2^{20} \approx 1.05 \times 10^6$ (MiB)	10^6
Giga	$2^{30} \approx 1.07 \times 10^9$ (GiB)	10^9

With n address bits you can name 2^n locations: 32 bits → 4 GiB; 48 bits → 256 TiB.

Exercises

1.1 Convert `11001010` to decimal and hex (unsigned).

1.2 Convert 200 to binary and hex.

1.3 Write -1 and -128 as 8-bit two's complement.

1.4 Compute $13 - 20$ in 8-bit two's complement. What is the result in decimal?

1.5 Store `0xA1B2C3D4` in little-endian order. List the four bytes from lowest to highest address.

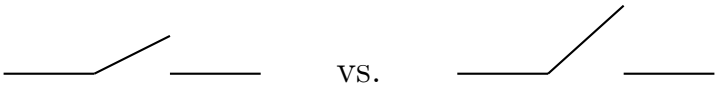
1.6 How many bytes can a 16-bit address bus reach? A 36-bit one?

Chapter 2

Transistors and Logic Gates

2.1 The transistor: a switch controlled by voltage

Everything in a computer is built from one component, the **transistor**. For our purposes a transistor is a tiny *switch with no moving parts*: a voltage on its **gate** terminal decides whether current can flow between its other two terminals.



gate = 1: **closed** — current flows

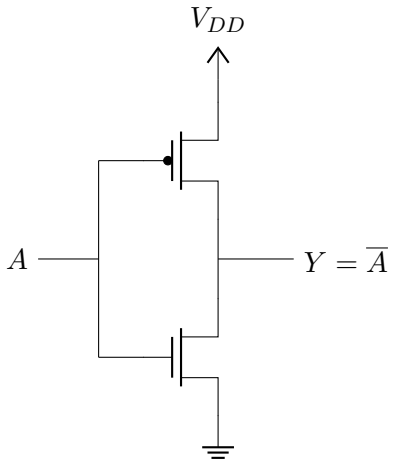
gate = 0: **open** — no current

There are two flavours, used in pairs. This is called **CMOS** (Complementary MOS) technology:

- **nMOS**: conducts when its gate is **1** (high voltage). Good at pulling an output down to 0.
- **pMOS**: conducts when its gate is **0**. Good at pulling an output up to 1.

2.1.1 The CMOS inverter (NOT gate)

Put a pMOS between the supply V_{DD} and the output, and an nMOS between the output and ground. Connect both gates to the input A.



Worked example 2.1: How the inverter works

1. Input $A = 0$: the pMOS is on, the nMOS is off. The output is connected to V_{DD}, so $Y = 1$.
2. Input $A = 1$: the pMOS is off, the nMOS is on. The output is connected to ground, so $Y = 0$.
3. In both cases exactly one transistor is on, so there is (almost) no current from V_{DD} to ground. This is why CMOS uses so little power when nothing changes: power is only burnt when signals *switch*.

2.1.2 NAND and NOR from four transistors

A 2-input **NAND** uses two nMOS in *series* (both must be on to pull the output down) and two pMOS in *parallel* (either being on pulls the output up). A **NOR** is the mirror image. Typical transistor counts: NOT = 2, NAND/NOR = 4, AND/OR = 6 (a NAND plus an inverter), XOR ≈ 8–12.

Key idea

NAND is universal: any logic function can be built using only NAND gates (so can NOR). This is why chips are full of NANDs.

2.2 Logic gates and truth tables

A gate takes bits in and gives a bit out. A **truth table** lists the output for every input combination. For n inputs there are 2^n rows.

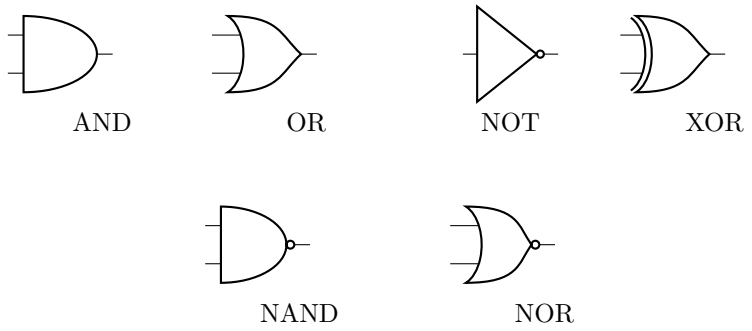


Figure 2.1: The standard gate symbols (the small circle means "inverted").

A	B	AND $A \cdot B$	OR $A + B$	XOR $A \oplus B$	NAND $\overline{A \cdot B}$	NOR $\overline{A + B}$	XNOR $\overline{A \oplus B}$
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

Intuition. AND = "both". OR = "at least one". XOR = "exactly one / they differ". NOT flips.

2.3 Boolean algebra

We write AND as multiplication, OR as addition and NOT with a bar. Useful laws:

Law	Rule
Identity	$A \cdot 1 = A,\ A + 0 = A$
Null	$A \cdot 0 = 0,\ A + 1 = 1$
Idempotent	$A \cdot A = A,\ A + A = A$
Complement	$A \cdot \overline{A} = 0,\ A + \overline{A} = 1$
Distributive	$A(B + C) = AB + AC$
De Morgan	$\overline{AB} = \overline{A} + \overline{B},\ \overline{A + B} = \overline{A} \cdot \overline{B}$

Worked example 2.2: Simplifying a circuit

Simplify $Y = AB + A\overline{B}$.

1. Factor out A: $Y = A(B + \overline{B})$.
2. By the complement law $B + \overline{B} = 1$.
3. So $Y = A \cdot 1 = A$.

Two AND gates, a NOT and an OR collapse into a plain wire. Simplification saves silicon, power and delay.

Worked example 2.3: Building NOT, AND, OR from NAND only

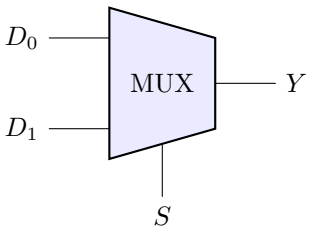
1. NOT: tie both inputs together: $\overline{A \cdot A} = \overline{A}$.
2. AND: NAND followed by that NOT: $\overline{\overline{AB}} = AB$.
3. OR: invert both inputs first, then NAND. By De Morgan, $\overline{\overline{A} \cdot \overline{B}} = A + B$.

2.4 Combinational building blocks

A circuit whose output depends *only on its present inputs* (no memory) is **combinational**.

2.4.1 Multiplexer (mux): the "if" of hardware

A 2-to-1 mux picks one of two inputs depending on a select bit S: $Y = \overline{S} \cdot D_0 + S \cdot D_1$.



Muxes appear everywhere: choosing the ALU result, choosing the next PC, choosing a register, forwarding data in the pipeline.

2.4.2 Decoder

A 2-to-4 decoder turns a 2-bit number into "one-hot" output: exactly one of four lines is 1.

A_1	A_0	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Memory uses decoders to turn an address into "select this row".

2.4.3 Adders

Half adder adds two bits: Sum $= A \oplus B$, Carry $= A \cdot B$.

Full adder adds two bits and an incoming carry C_{in}:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + C_{in}(A \oplus B).$$

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Notice: S and C_{out} together just encode *how many inputs are 1* (0,1,2,3) in binary.

Ripple-carry adder. Chain n full adders; each C_{out} feeds the next C_{in}.

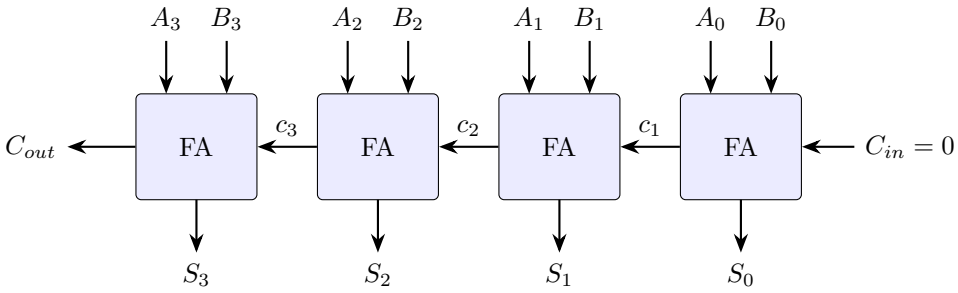


Figure 2.2: A 4-bit ripple-carry adder. The carry "ripples" from right to left.

Worked example 2.4: Adding 6 + 7 with a 4-bit adder

$A = 0110$ (6), $B = 0111$ (7), start with $C_{in} = 0$.

1. Bit 0: $0 + 1 + 0$: sum 1, carry 0.
2. Bit 1: $1 + 1 + 0$: sum 0, carry 1.
3. Bit 2: $1 + 1 + 1$: sum 1, carry 1.
4. Bit 3: $0 + 0 + 1$: sum 1, carry 0.
5. Result bits (3 down to 0): `1101` $= 13$. ✓

Why speed matters. If one full adder takes 2 gate delays to produce its carry, a 32-bit ripple adder needs about $32 \times 2 = 64$ gate delays in the worst case. Real CPUs use **carry-lookahead** or **prefix** adders that compute all carries in parallel in about $\log_2 n$ steps (≈ 5 levels for 32 bits), trading extra gates for much less delay.

2.4.4 The ALU (Arithmetic Logic Unit)

The ALU computes several functions in parallel and a mux selects the one we want.

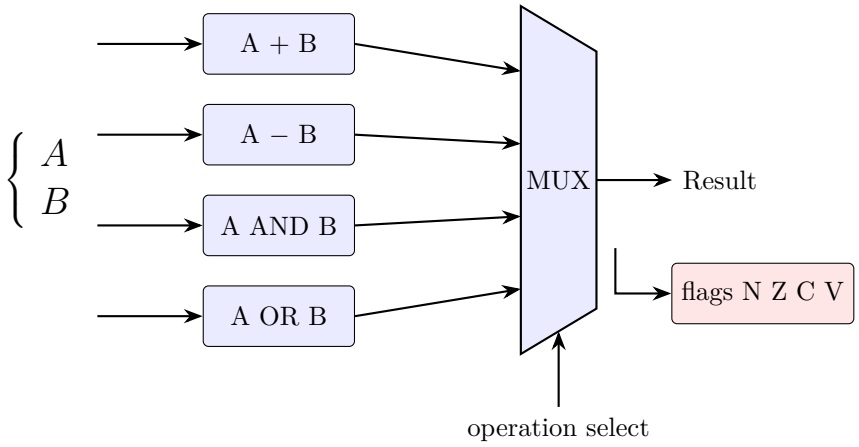


Figure 2.3: A simplified ALU: all operations are computed, one result is selected. The *flags* describe the result (negative, zero, carry, overflow).

Exercises

2.1 Build the truth table of $Y = \overline{A}\,B + A\,\overline{B}$. Which standard gate is it?

2.2 Use De Morgan to rewrite $\overline{A + \overline{B}}$ with only AND and NOT.

2.3 Simplify $Y = A + \overline{A}B$. (Hint: write $A = A(1 + B)$.)

2.4 Add `1011` and `0110` as a 4-bit adder. What carry-out do you get, and what does it mean for unsigned numbers?

2.5 How many transistors are needed for a 32-bit ripple-carry adder if one full adder uses about 28 transistors?

2.6 Draw a 4-to-1 mux using three 2-to-1 muxes.

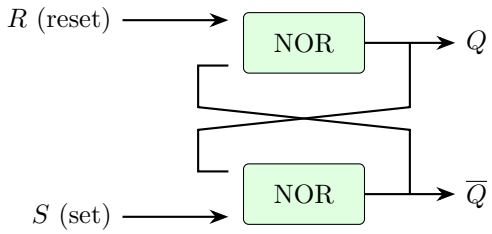
Chapter 3

Memory Cells and the Clock

Gates alone can compute but cannot *remember*. A CPU must hold a value (the program counter, a register) and then update it at a precisely chosen moment. This chapter introduces memory built from gates and the **clock** that orchestrates everything.

3.1 From gates to memory: feedback

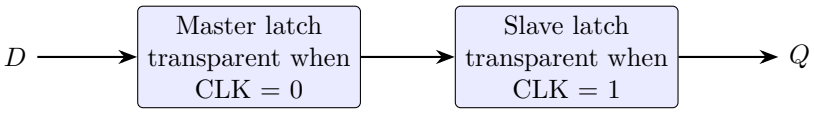
The trick is **feedback**: feed a gate's output back into its own input. Two NOR gates connected in a loop form an **SR latch**.



S	R	Effect on Q
0	0	**holds** its previous value (memory!)
1	0	set: $Q = 1$
0	1	reset: $Q = 0$
1	1	forbidden (both outputs would be 0)

3.1.1 D latch and D flip-flop

We want one data input D and one control input. A **D latch** is transparent (output follows D) while its enable is high and holds when it is low. Two latches in series with opposite enables give a **D flip-flop** (master–slave):



Key idea

A **D flip-flop** copies D to Q *only at the rising edge of the clock* and ignores D the rest of the time. It is the fundamental 1-bit storage element of a CPU. A **register** is simply n flip-flops sharing one clock (e.g. 32 for an ARM register).

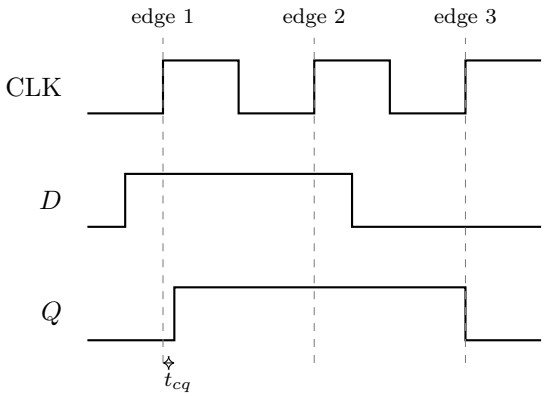
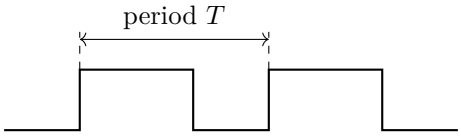


Figure 3.1: Timing diagram of a D flip-flop. At edge 1, $D = 1$ so Q becomes 1 a little later (t_{cq}). At edge 3, $D = 0$, so Q falls. Changes of D between edges are ignored.

3.2 The clock

The **clock** is a square wave that tells the whole chip "now!". All flip-flops update at the same instant, so the chip behaves like a synchronised orchestra.



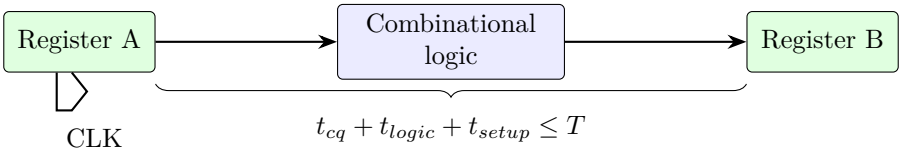
$$f = \frac{1}{T} \qquad T = \frac{1}{f}$$

Worked example 3.1: Frequency and period

1. A 1 GHz clock: $T = 1/10^9\,\text{s} = 1\,\text{ns}$.
2. A 3 GHz clock: $T = 1/(3 \times 10^9) \approx 0.333\,\text{ns} = 333\,\text{ps}$.
3. In 333 ps light travels only $3 \times 10^8 \times 333 \times 10^{-12} \approx 10\,\text{cm}$ in vacuum, and signals on a chip are slower. This is why a fast chip must be small.

3.2.1 The timing rule: setup, hold and the critical path

Between two registers sits combinational logic. Each clock tick, data leaves register A, passes through logic, and must arrive at register B *before the next tick*.



Three timing quantities of a flip-flop:
t_{cq} **(clock-to-Q)** delay from the clock edge until the new value appears at Q.
t_{setup} time the data must be stable *before* the edge.
t_{hold} time the data must stay stable *after* the edge.
The **setup constraint** gives the minimum clock period:

$$T \geq t_{cq} + t_{logic,\max} + t_{setup}.$$

The slowest path between any two registers is the **critical path**; it sets the clock speed of the whole chip.

Worked example 3.2: Maximum clock frequency

Registers have $t_{cq} = 50$ ps and $t_{setup} = 30$ ps. The slowest logic path takes 400 ps.

1. $T_{min} = 50 + 400 + 30 = 480$ ps.
2. $f_{max} = 1/480\,\text{ps} \approx 2.08$ GHz.
3. If we try to run at 3 GHz ($T = 333$ ps), the data arrives too late. B captures garbage (a *setup violation*).

Worked example 3.3: Speeding up by cutting the path (pipelining preview)

Insert a register in the middle of the 400 ps logic so each half takes 200 ps.

1. New period: $T = 50 + 200 + 30 = 280$ ps, so $f \approx 3.57$ GHz.
2. Latency of one operation: now 2 cycles = 560 ps (slightly *worse* than 480 ps).
3. But a new operation can start every 280 ps, so **throughput** improves from 1/480 to 1/280 per ps: 1.71× faster.

This is the whole idea of the pipeline (Chapter 6).

Careful!

Hold violations cannot be fixed by lowering the clock speed. If logic is *too fast*, new data races through and overwrites the value the next register was supposed to capture. Designers add delay buffers to fix them.

3.2.2 Why clock speed stopped growing

Dynamic power in CMOS is

$$P \approx \alpha\, C\, V^2 f$$

where α is the fraction of gates switching, C the capacitance, V the voltage and f the frequency.

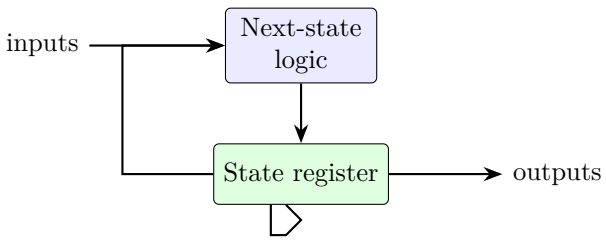
Worked example 3.4: The cost of speed

1. Lower voltage by 20% ($V \to 0.8V$): power is multiplied by $0.8^2 = 0.64$ (a 36% saving).
2. Raise frequency by 20% *and* increase voltage by 10% to make it stable: power $\times 1.2 \times 1.1^2 = 1.2 \times 1.21 = 1.45$. For 20% more speed you pay 45% more power!

Around 2005 chips hit the **power wall** (heat could not be removed), which is why modern CPUs sit at 3–5 GHz and add more *cores* instead.

3.3 Finite state machines: a counter

A circuit made of registers plus combinational logic is a **finite state machine (FSM)**. The registers hold the current *state*; logic computes the *next state*.



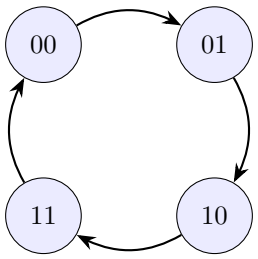
Worked example 3.5: A 2-bit counter

We want the sequence $00 \to 01 \to 10 \to 11 \to 00 \to \dots$ on each clock tick. Let the state bits be $Q_1 Q_0$.

1. Write the table of current $\to$ next state:

Q_1	Q_0	D_1	D_0
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

2. Read the columns: $D_0 = \overline{Q_0}$ and $D_1 = Q_1 \oplus Q_0$.
3. Hardware: two flip-flops, one NOT gate, one XOR gate. Each tick, the flip-flops load $D_1 D_0$.



The **program counter (PC)** of a CPU is exactly this: a 32-bit register that adds 4 each clock tick (unless a branch changes it).

3.4 Registers, register files and the first memory

A **register file** is a small array of registers with read ports and a write port. ARM has 16 visible registers of 32 bits. It is built with decoders and muxes (Chapter 4). Larger memories (cache, RAM) use denser cells (SRAM, DRAM) and are covered in Chapter 7.

Exercises

3.1 A CPU runs at 4.2 GHz. What is the clock period in picoseconds?

3.2 $t_{cq} = 40$ ps, $t_{setup} = 20$ ps and the critical path logic is 340 ps. Compute f_{max}.

3.3 Split that logic into two equal stages. New f_{max}? New latency of an operation? New throughput speed-up?

3.4 A chip dissipates 60 W at 1.0 V and 3 GHz. Estimate the power at 0.9 V and 2.7 GHz (use $P \propto V^2 f$).

3.5 Complete a table for a 3-bit counter (000 to 111 to 000) and derive D_2.

3.6 Why can't an SR latch be used directly as a register in a synchronous CPU?

Chapter 4

The CPU: Datapath and Control

We now have gates, adders, muxes and registers. Time to assemble a processor.

4.1 The five ingredients

Component	Role
PC (program counter)	Register holding the address of the next instruction.
Instruction memory	Gives the 32-bit instruction stored at address PC.
Register file	16 fast 32-bit registers; reads two and writes one per cycle.
ALU	Does the arithmetic and logic.
Data memory	Loads and stores data (in reality: the cache hierarchy).
Control unit	Decodes the instruction and tells everyone else what to do.

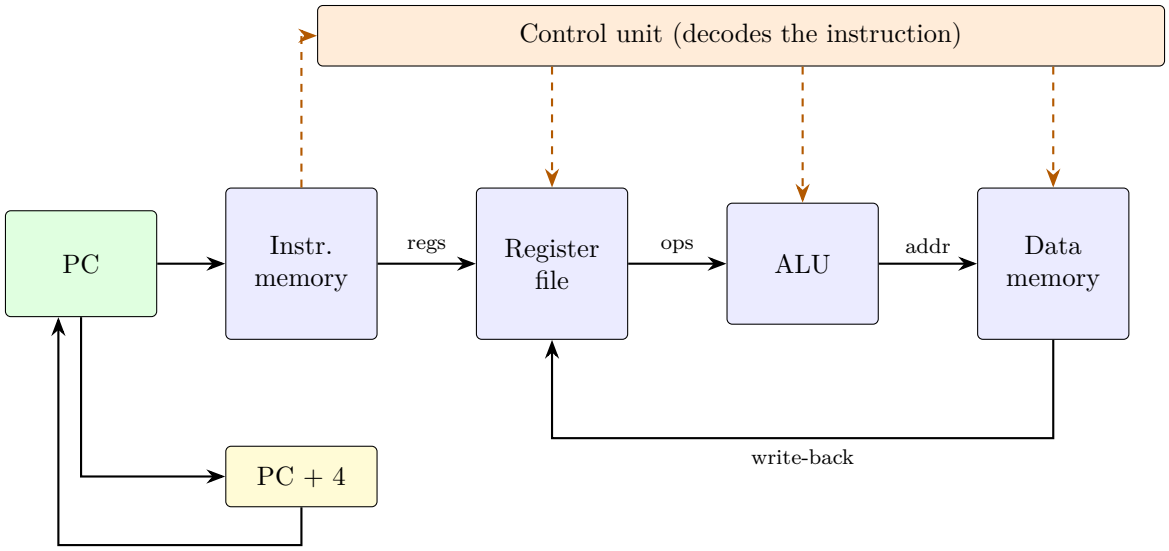
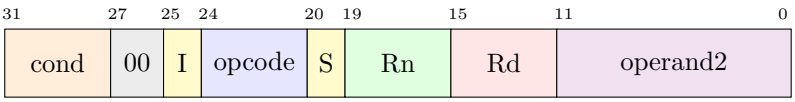


Figure 4.1: A simplified datapath. Solid arrows carry data; dashed orange arrows are control signals produced by the control unit.

4.2 What does an instruction look like?

An ARMv7 instruction is exactly 32 bits. For the common *data-processing* instructions (ADD, SUB, MOV, AND…) the layout is:



- **cond** (4 bits): the instruction only executes if the flags satisfy this condition (`1110` = always).
- **I**: 1 if operand2 is an immediate constant, 0 if it is a register.
- **opcode**: which operation (`0100` = ADD, `0010` = SUB, `1101` = MOV...).
- **S**: 1 means "also update the flags".
- **Rn**, **Rd**: first source register and destination register.

Worked example 4.1: Encoding `ADD r0, r1, r2`

We need r0 ← r1 + r2.

1. cond = always = `1110`.
2. bits 27–26 = `00`; I = `0` (operand2 is a register).
3. opcode ADD = `0100`; S = `0`.
4. Rn = r1 = `0001`; Rd = r0 = `0000`.
5. operand2 = no shift (`00000000`) followed by Rm = r2 = `0010`: `000000000010`.
6. Concatenate: `1110 0000 1000 0001 0000 0000 0000 0010` = `0xE0810002`.

That 32-bit number *is* the instruction in memory.

Worked example 4.2: Encoding `MOV r0, #5`

1. cond `1110`, `00`, I = `1` (immediate), opcode MOV = `1101`, S = `0`.
2. Rn is unused: `0000`. Rd = r0 = `0000`.
3. operand2 = rotation `0000` and 8-bit value `00000101`.
4. Result: `1110 0011 1010 0000 0000 0000 0000 0101` = `0xE3A00005`.

Key idea

There is no magic: "machine code" is just numbers. The control unit looks at the bit fields, and those bits steer muxes and enable lines in the datapath.

4.3 Fetch, decode, execute: one instruction in detail

Assume the program is stored in memory like this and r1 = 10, r2 = 32:

Address	Machine code	Assembly
`0x8000`	`0xE0810002`	`ADD r0, r1, r2`
`0x8004`	`0xE5903000`	`LDR r3, [r0]`

Worked example 4.3: Tracing `ADD r0, r1, r2`

Initially PC = `0x8000`.

1. **Fetch**: send PC to the instruction memory. Out comes `0xE0810002`. Meanwhile an adder computes PC+4 =`0x8004`.
2. **Decode**: the control unit reads the fields: opcode = ADD, Rn = 1, Rm = 2, Rd = 0.
3. **Register read**: the register file outputs r1 = 10 and r2 = 32 at the same time (two read ports).
4. **Execute**: the ALU is told to add: 10 + 32 = 42.
5. **Memory**: nothing to do (not a load/store).
6. **Write-back**: 42 is written into r0 at the next clock edge. The PC register also loads `0x8004`.

Worked example 4.4: Tracing `LDR r3, [r0]` (a load)

Now PC = `0x8004`, r0 = 42 =`0x2A`. Assume memory at address 42 holds the value 99.

1. Fetch `0xE5903000`; PC←`0x8008`.
2. Decode: "load word, base register r0, offset 0, destination r3".
3. Read r0 = 42.
4. ALU computes the address: $42 + 0 = 42$.
5. **Memory**: read the word at address 42 ⇒ 99.
6. Write-back: r3 ← 99 (the mux in front of the register file chooses the memory output rather than the ALU output).

4.4 Control signals

The control unit produces a handful of bits for each instruction:

Instruction type	RegWrite	ALUSrc	MemRead/Write	MemToReg
Data-proc (register)	1	reg	none	ALU
Data-proc (immediate)	1	imm	none	ALU
LDR (load)	1	imm (offset)	read	memory
STR (store)	0	imm (offset)	write	–
B (branch)	0	–	none	– (PC updated)

For a **branch**, a mux in front of the PC selects "PC + offset" instead of "PC + 4".

4.5 RISC versus CISC

	RISC (ARM, RISC-V)	CISC (x86)
Instruction size	fixed (32 bits; 16 in Thumb)	variable (1–15 bytes)
Memory access	only LDR/STR touch memory (*load/store architecture*)	many instructions can read memory
Decoding	simple, fast	complex (x86 chips translate to internal micro-ops)
Registers	many (16–32)	fewer (16 in x86-64)

RISC regularity makes pipelining easy, which is a big reason why ARM is efficient in phones.

4.6 Why the single-cycle CPU is too slow

In the single-cycle design the clock period must fit the *slowest* instruction (a load, which passes through every block). Typical delays: instruction fetch 200 ps, register read 100 ps, ALU 200 ps, data memory 200 ps, register write 100 ps.

$$T_{load} = 200 + 100 + 200 + 200 + 100 = 800 \text{ ps} \;\Rightarrow\; f \approx 1.25 \text{ GHz}.$$

Even an ADD, which needs only 600 ps, must wait 800 ps. The fix is to overlap instructions (Chapter 6).

Exercises

4.1 Encode `SUB r4, r5, r6` (opcode SUB = `0010`). Give the hex word.

4.2 Decode `0xE2832001`: which instruction is it? (Hint: I = 1, opcode `0100`, Rn = 3, Rd = 2, imm = 1.)

4.3 Why does a load/store architecture have fewer, simpler instruction formats?

4.4 The program counter holds `0x9000`. After three sequential ARM instructions, what is it?

4.5 In the load trace, which control signal selects whether the register file receives the ALU output or the memory output?

4.6 A fetch needs 200 ps, decode+reg read 100 ps, ALU 200 ps. Which instruction types can skip the memory stage, and how much time is wasted in a single-cycle design?

Chapter 5

ARMv7 Assembly Language

Assembly language is the human-readable form of machine code: one line of assembly equals one 32-bit instruction. Learning it shows you exactly what the hardware of Chapter 4 does.

5.1 The programmer's view of an ARM core

5.1.1 Registers

There are 16 registers visible to the programmer, each 32 bits wide.

`r0`	argument 1 and return value
`r1`	argument 2
`r2`	argument 3
`r3`	argument 4
`r4`	callee-saved variable
`r5`	
`r6`	
`r7`	(syscall number on Linux)
`r8`	
`r9`	
`r10`	
`r11`	frame pointer (fp)
`r12`	scratch (ip)
`r13`	**sp** stack pointer
`r14`	**lr** link register (return address)
`r15`	**pc** program counter

Figure 5.1: ARMv7 general-purpose registers. Blue: arguments (may be overwritten by a call). Green: preserved across calls.

5.1.2 The status register CPSR and its flags

Four bits record facts about the last flag-setting operation:

Flag	Name	Meaning
N	Negative	result's top bit is 1 (negative if signed)
Z	Zero	result is exactly 0
C	Carry	unsigned carry out (for subtraction: no borrow)
V	oVerflow	signed result did not fit in 32 bits

Worked example 5.1: Flags after a few additions (32-bit)

Instructions with the `S` suffix (`ADDS`, `SUBS`, `CMP`...) update the flags.

1. `0x7FFFFFFF` + 1 = `0x80000000`: top bit set, so N=1. Not zero, Z=0. No carry out, C=0. Two positives gave a negative, so V=1 (signed overflow).
2. `0xFFFFFFFF` + 1 = `0x00000000` with a carry out: N=0, Z=1, C=1, V=0. (As signed numbers this is $-1+1=0$, perfectly fine, no overflow.)
3. $5-5$: result 0, so Z=1, N=0.

5.2 Data-processing instructions

General form: `OP{S}{cond} Rd, Rn, Operand2` meaning $Rd \leftarrow Rn$ OP Operand2.

```
    mov   r0, #10          @ r0 = 10
    mov   r1, r0           @ r1 = r0
    add   r2, r0, r1       @ r2 = r0 + r1 = 20
    sub   r3, r2, #4       @ r3 = r2 - 4 = 16
    rsb   r4, r3, #100     @ r4 = 100 - r3 (reverse subtract) = 84
    mul   r5, r0, r1       @ r5 = r0 * r1 = 100
    and   r6, r2, #0xF     @ r6 = r2 & 0xF = 20 & 15 = 4
    orr   r7, r6, #0x10    @ r7 = r6 | 16 = 20
    eor   r8, r7, r7       @ r8 = 0 (xor with itself)
    mvn   r9, #0           @ r9 = NOT 0 = 0xFFFFFFFF = -1
    bic   r10, r2, #4      @ r10 = r2 AND NOT 4 (clear bit 2)
```

5.2.1 The barrel shifter: a free shift on operand 2

The second source operand can be shifted *for free* within the same instruction. Shifts: `LSL` (logical left), `LSR` (logical right), `ASR` (arithmetic right, keeps sign), `ROR` (rotate).

Worked example 5.2: Multiplying by 5 with no multiplier

Goal: r0 $\leftarrow 5\times$ r1, with r1 = 7.

1. $5x = x + 4x$ and $4x = x \ll 2$.
2. `add r0, r1, r1, LSL #2` $\Rightarrow$ r0 $= 7 + (7 \ll 2) = 7 + 28 = 35$. ✓

One instruction, one cycle.

Operation	Instruction	Effect on x = `0000 0110` (6)
$\times 2^k$	`lsl r0, r1, #2`	`0001 1000` = 24
$\div 2^k$ unsigned	`lsr r0, r1, #1`	`0000 0011` = 3
$\div 2^k$ signed	`asr r0, r1, #1`	same, but a negative stays negative

5.2.2 Immediates: the 8-bit rotated constant rule

An instruction has only 12 bits for a constant: an 8-bit value rotated right by an even number of positions. So `0xFF`, `0xFF00`, `0x3FC` are encodable; `0x101` (needs 9 significant bits in a row) is not. For arbitrary 32-bit constants use:

```
    movw r0, #0x5678       @ r0 = 0x00005678 (low 16 bits, clears the
    top)
    movt r0, #0x1234       @ r0 = 0x12345678 (writes the top 16 bits)
    ldr  r1, =0x12345678 @ pseudo-instruction: assembler picks the
    best way
```

5.3 Comparisons and conditional execution

`CMP Rn, Op` computes $Rn - Op$, *throws the result away* and keeps only the flags. `TST` does the same with AND. Any instruction can be made conditional by a suffix:

Suffix	Meaning	Flags tested
EQ	equal	Z=1
NE	not equal	Z=0
GT / GE	greater (or equal)	signed
LT / LE	less (or equal)	signed
HI / HS(CS)	higher (or same)	unsigned
LO(CC) / LS	lower (or same)	unsigned
MI / PL	negative / positive	N
VS / VC	overflow / no overflow	V
AL	always (default)	–

Careful!

Signed and unsigned comparisons use *different* conditions. After `cmp r0, r1` with r0 = `0xFFFFFFFF` and r1 = 1: unsigned, $4\,294\,967\,295 > 1$ so `HI` is true. Signed, $-1 < 1$ so `LT` is true. Same bits, different meaning (see Chapter 1).

Worked example 5.3: If / else without a branch

C code: `max = (a > b) ? a : b;` with a in r0, b in r1.

```
1     cmp     r0, r1          @ compute a - b, set flags
2     movgt   r2, r0          @ if a > b : max = a
3     movle   r2, r1          @ else     : max = b
```

Check with a=9, b=4: $9-4>0$ so GT is true, r2 = 9; the LE move is skipped. ✓Conditional execution avoids a branch, and branches can be costly in a pipeline (Chapter 6).

5.4 Branches and loops

- `B label`: jump (PC ← label). `BEQ`, `BNE`, `BLT`...: conditional jumps.
- `BL label`: jump and save the return address in `lr` (function call).
- `BX lr`: jump to the address in `lr` (function return).

Worked example 5.4: A loop: factorial

C: `r = 1; while (n > 1) { r = r * n; n-; }` with n in r0.

```
1  fact:   mov   r1, #1          @ r1 = result = 1
2  loop:   cmp   r0, #1          @ compare n with 1
3          ble   done            @ if n <= 1 leave the loop
4          mul   r1, r1, r0      @ result = result * n
5          sub   r0, r0, #1      @ n = n - 1
6          b     loop
7  done:   mov   r0, r1          @ return value in r0
8          bx    lr              @ return to caller
```

Trace for $n = 4$:

iteration	r0 (n)	r1 (result)	what happens
start	4	1	
1	3	4	1×4, then n–
2	2	12	4×3
3	1	24	12×2
4	1	24	`cmp 1,1`: LE true, exit

Return value r0 = 24 = 4!. ✓

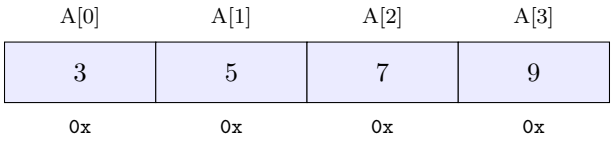
5.5 Memory access: load and store

Only `LDR` (load) and `STR` (store) touch memory. Variants: `LDRB/STRB` (byte), `LDRH/STRH` (halfword, 16 bits).

Syntax	Meaning
`ldr r0, [r1]`	r0 ← mem[r1]
`ldr r0, [r1, #4]`	r0 ← mem[r1+4] (offset)
`ldr r0, [r1, #4]!`	r1 ← r1+4, *then* load from r1 (pre-index)
`ldr r0, [r1], #4`	load from r1, *then* r1 ← r1+4 (post-index)
`ldr r0, [r1, r2, LSL #2]`	r0 ← mem[r1 + 4·r2] (array indexing)
`str r0, [r1]`	mem[r1] ← r0

5.5.1 Arrays in memory

An array of four 32-bit integers {3, 5, 7, 9} starting at address `0x2000`:



Because each integer takes 4 bytes, the address of A[i] is base + $4i$, which is exactly what the barrel shifter gives: `[r1, r2, LSL #2]`.

Worked example 5.5: Summing an array

C: `sum = 0; for (i = 0; i < n; i++) sum += A[i];` r0 = address of A, r1 = n.

```
1  sum:    mov   r2, #0              @ r2 = sum
2  again:  cmp   r1, #0              @ any elements left?
3          beq   end
4          ldr   r3, [r0], #4        @ r3 = *r0 ; r0 += 4  (post-
       index)
5          add   r2, r2, r3          @ sum += r3
6          sub   r1, r1, #1          @ n--
7          b     again
8  end:    mov   r0, r2              @ return sum
9          bx    lr
```

Trace with A = {3, 5, 7, 9} at `0x2000`, n = 4:

after iteration	r0 (pointer)	r3 (loaded)	r2 (sum)	r1 (left)
0	0x2000	–	0	4
1	0x2004	3	3	3
2	0x2008	5	8	2
3	0x200C	7	15	1
4	0x2010	9	24	0

The function returns 24. ✓

5.5.2 Multiple registers and the stack

`PUSH {r4, lr}` and `POP {r4, pc}` save and restore several registers at once. The stack lives in RAM and **grows downward** (towards smaller addresses); `sp` points to the last item pushed.

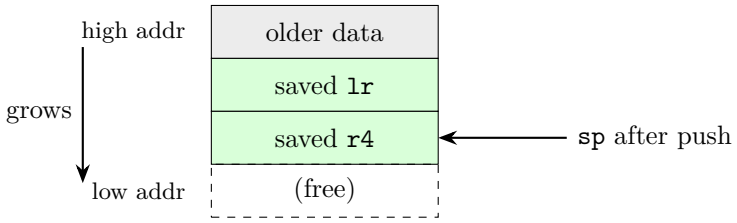


Figure 5.2: After `push {r4, lr}`: sp decreases by 8; `r4` is stored at the lower address, `lr` just above it.

5.6 Functions and the calling convention

Everyone must agree on how to pass arguments, otherwise code from different compilers could not work together. ARM's convention (AAPCS):

- Arguments in `r0-r3` (more go on the stack). Result in `r0`.
- The callee may freely change `r0-r3` and `r12`.
- The callee must preserve `r4-r11` and `sp`: if it uses them, it saves and restores them.
- `BL` stores the return address in `lr`. A function that calls another must save `lr` first, or it loses its own way home.

Worked example 5.6: Recursive factorial

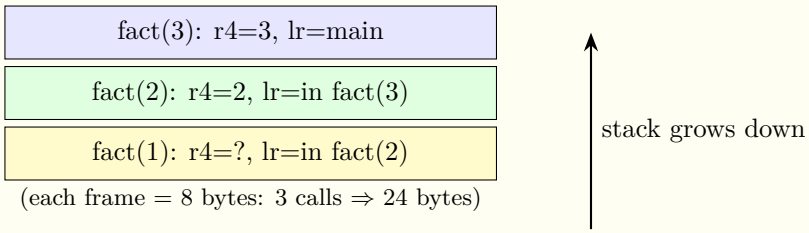
C: `int fact(int n) { return n <= 1 ? 1 : n * fact(n-1); }`

```
1  fact:   push  {r4, lr}        @ save r4 and the return address
2          cmp   r0, #1
3          ble   base            @ if n <= 1 go to base case
4          mov   r4, r0          @ remember n in r4 (callee-saved)
5          sub   r0, r0, #1      @ argument n-1
6          bl    fact            @ r0 = fact(n-1)
7          mul   r0, r0, r4      @ r0 = fact(n-1) * n
8          pop   {r4, pc}        @ restore r4 AND return (pc = saved
       lr)
9  base:   mov   r0, #1
10         pop   {r4, pc}
```

Trace of `fact(3)`:

1. fact(3): push, r4 ← 3, call fact(2).
2. fact(2): push, r4 ← 2, call fact(1).
3. fact(1): base case, r0 ← 1, pop and return.

4. Back in fact(2): r0 $= 1 \times r4 = 1 \times 2 = 2$, pop and return.
5. Back in fact(3): r0 $= 2 \times 3 = 6$, pop and return. ✓



The stack arrow above points up only to indicate older frames at the top; addresses decrease going down.

Worked example 5.7: Euclid's GCD using conditional execution

gcd(a, b): while $a \neq b$, subtract the smaller from the larger. a in r0, b in r1.

```
1  gcd:    cmp    r0, r1
2          subgt  r0, r0, r1      @ if a > b : a = a - b
3          sublt  r1, r1, r0      @ if a < b : b = b - a
4          bne    gcd             @ repeat until a == b
5          bx     lr              @ answer in r0
```

Trace for $(12, 18)$: $12 < 18 \Rightarrow b = 6$; $12 > 6 \Rightarrow a = 6$; now $6 = 6$, exit. Result 6. ✓
(Note: `SUBGT`/`SUBLT` have no `S`, so they do not touch the flags set by `CMP`; `BNE` still sees that result.)

5.7 A complete program: Hello, world on Linux

```
1          .data
2  msg:    .ascii "Hello, world!\n"
3  len =   . - msg                 @ length of the string (14)
4
5          .text
6          .global _start
7  _start: mov    r0, #1           @ file descriptor 1 = stdout
8          ldr    r1, =msg         @ address of the text
9          mov    r2, #len         @ number of bytes
10         mov    r7, #4           @ syscall number 4 = write
11         svc    #0               @ supervisor call: ask the OS kernel
12         mov    r0, #0           @ exit code 0
13         mov    r7, #1           @ syscall number 1 = exit
14         svc    #0
```

Build and run on an ARM Linux machine (or with QEMU): `as -o hello.o hello.s && ld -o hello hello.o && ./hello`.

Key idea

A program cannot print by itself: it uses `SVC` to enter the operating system, which runs in a privileged mode and owns the hardware. This is the boundary between user programs and the kernel.

5.8 What the compiler really does

C	ARM
`c = a + b;`	`add r2, r0, r1`
`x = a * 4 + b;`	`add r2, r1, r0, lsl #2`
`v = A[i];`	`ldr r3, [r0, r1, lsl #2]`
`p->next` (offset 8)	`ldr r0, [r0, #8]`
`if (a == 0) {...}`	`cmp r0, #0` then `bne skip`
`f(x);`	`mov r0, rX; bl f`

You can see real compiler output with `gcc -S -O1 -march=armv7-a file.c`, or on godbolt.org.

5.9 Thumb, NEON and beyond

- **Thumb/Thumb-2**: a compressed mix of 16- and 32-bit instructions for smaller code.
- **NEON**: 128-bit SIMD registers (q0–q15) that process 4 floats or 16 bytes at once. For example, `vadd.f32 q0, q1, q2` performs four additions in one instruction (see Chapter 9).
- **AArch64 (ARMv8+)**: 64-bit ISA with 31 general registers. Concepts carry over directly.

Exercises

5.1 What are r0 and the flags after `mov r0, #0x7FFFFFFF; adds r0, r0, #1`?

5.2 Write ARM code for $r1 = 10 \cdot r0$ using shifts and adds only. (Hint: $10x = 8x + 2x$.)

5.3 Write a function that returns the absolute value of r0 using `cmp`/`rsblt`.

5.4 Write a loop that sets the first 10 words of an array at r0 to zero.

5.5 Trace `fact(4)` through the recursive version; how many bytes of stack are used at the deepest point?

5.6 Why must a function that calls `bl` save `lr`?

5.7 Explain the difference between `ldr r0, [r1, #4]!` and `ldr r0, [r1], #4`.

Chapter 6

Performance and Pipelining

6.1 How do we measure speed?

The central equation of computer architecture (the "iron law") is

$$\text{CPU time} = \underbrace{\text{Instruction count}}_{\text{compiler, ISA}} \times \underbrace{\text{CPI}}_{\text{microarchitecture}} \times \underbrace{T_{clk}}_{\text{circuit, technology}}$$

where CPI is the average number of clock cycles per instruction.

Worked example 6.1: Using the iron law

A program executes 2×10^9 instructions on a 3 GHz CPU with CPI $= 1.5$.

1. $T_{clk} = 1/(3 \times 10^9)$ s.
2. Time $= 2 \times 10^9 \times 1.5 \times \dfrac{1}{3 \times 10^9} = 1.0$ s.
3. If a better pipeline lowers CPI to 1.0 (same clock, same program): time $= 0.67$ s, a $1.5\times$ speed-up.

Careful!

Clock frequency alone says nothing. A 2 GHz chip with CPI 0.5 (executing 2 instructions per cycle) beats a 4 GHz chip with CPI 3.

6.1.1 Amdahl's law

If a fraction p of the work can be sped up by a factor N, the overall speed-up is

$$S = \frac{1}{(1-p) + p/N}.$$

Worked example 6.2: The limit of parallelism

80% of a program parallelises ($p = 0.8$).

1. $N = 8$ cores: $S = 1/(0.2 + 0.8/8) = 1/0.3 = 3.33$.
2. $N = 100$: $S = 1/(0.2 + 0.008) = 4.81$.
3. $N \to \infty$: $S = 1/0.2 = 5$. The serial 20% limits you to $5\times$ no matter how many cores.

Key idea

Speed up the common case, and remember that the part you did *not* speed up becomes the bottleneck.

6.2 The pipeline idea

Analogy

Washing, drying and folding laundry: one load takes 3 hours if you wait for each load to finish. But as soon as load 1 moves to the dryer, load 2 can enter the washer. Each load still takes 3 hours (latency) but a load finishes every hour (throughput).

We split instruction execution into 5 stages, each taking about one clock cycle:

	Stage	Work done
IF	Instruction Fetch	read instruction at PC; PC ← PC+4
ID	Instruction Decode	decode; read registers
EX	Execute	ALU operation or address calculation
MEM	Memory access	load or store data
WB	Write Back	write the result into a register

Between stages there are pipeline registers (flip-flops) that hold the partial work.

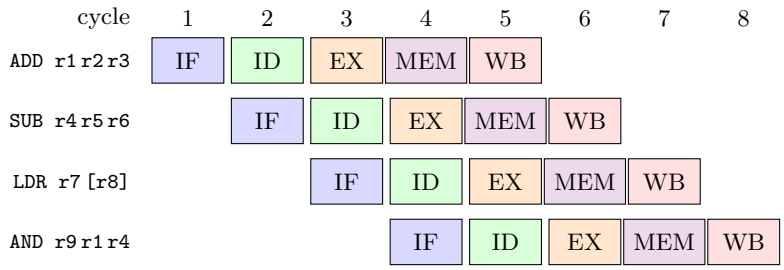


Figure 6.1: Ideal pipeline: once full, one instruction completes every cycle (CPI → 1).

Worked example 6.3: How much faster is a pipeline?

Stage delays: IF 200 ps, ID 150 ps, EX 200 ps, MEM 250 ps, WB 100 ps. Run 1000 instructions.

1. **Single-cycle**: period = 200+150+200+250+100 = 900 ps. Time = 1000×900 = 900 000 ps = 900 ns.
2. **Pipelined**: period = slowest stage = 250 ps.
3. Time = $(5 + 1000 - 1) \times 250 = 1004 \times 250 = 251\,000$ ps = 251 ns. (The first instruction needs 5 cycles; each later one finishes 1 cycle after the previous.)
4. Speed-up = $900/251 \approx 3.6$.

Not 5×, because the stages are *unbalanced* (the 250 ps stage sets the period). Real CPUs balance stages and use 10–20 (or more) stages.

6.3 Hazards: when the pipeline cannot flow

A **hazard** is a situation where the next instruction cannot proceed in the next cycle.

6.3.1 Structural hazards

Two instructions need the same hardware in the same cycle (e.g. one memory for instructions *and* data). Solution: duplicate resources (separate instruction and data L1 caches, which is why L1 is "split").

6.3.2 Data hazards

```
1     add   r1, r2, r3      @ writes r1 in WB (cycle 5)
2     sub   r4, r1, r5      @ wants r1 in EX  (cycle 4) -- too early!
```

Without any trick, `sub` would have to wait 2 cycles (stall). The fix is **forwarding** (bypassing): a wire sends the ALU result directly from the end of EX of `add` to the input of EX of `sub`, without waiting for the register write.

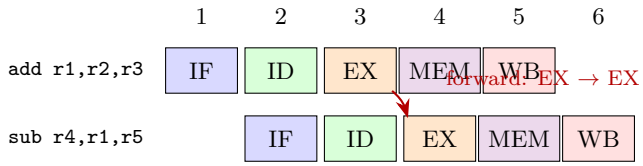


Figure 6.2: With forwarding, `sub` gets the result right after `add`'s EX stage: no stall.

Load-use hazard. A load only has its data after MEM, one stage later than the ALU result:

```
1     ldr   r1, [r0]        @ data available at the end of MEM
2     add   r2, r1, r3      @ needs r1 at the start of EX
```

Even with forwarding, one **bubble** (stall cycle) is unavoidable. Compilers try to put an independent instruction between the load and its use.

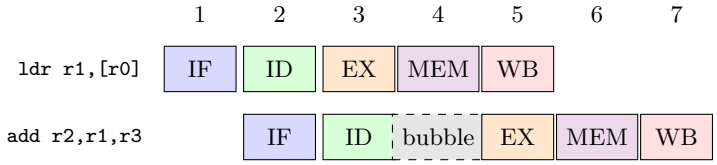
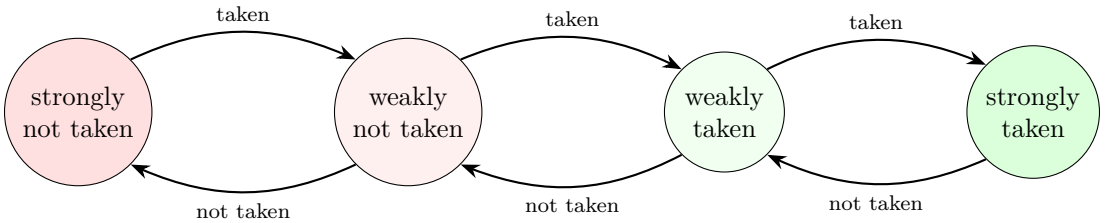


Figure 6.3: The load-use bubble costs one cycle.

6.3.3 Control hazards: branches

The pipeline fetches the next instruction before it knows whether the branch is taken. If the branch resolves in stage EX (cycle 3), two wrongly-fetched instructions must be thrown away (**flushed**): a penalty of 2 cycles in this short pipeline, and 15–20 cycles in deep modern pipelines.

Branch prediction. The CPU *guesses* the outcome and fetches speculatively. A classic predictor is a **2-bit saturating counter** per branch:



A loop branch that is taken 99 times then falls through once is mispredicted only once or twice per loop, because one surprise does not flip a "strong" state. Modern CPUs use far more elaborate predictors (history-based, neural) achieving 95–99% accuracy.

Worked example 6.4: Cost of mispredictions

20% of instructions are branches, the predictor is 95% accurate, a mispredict costs 15 cycles, the ideal CPI is 1.

1. Mispredictions per instruction: $0.20 \times 0.05 = 0.01$.
2. Extra cycles per instruction: $0.01 \times 15 = 0.15$.
3. CPI $= 1 + 0.15 = 1.15$. A 5% drop in accuracy to 90% would give $1 + 0.2 \times 0.1 \times 15 = 1.30$.

6.4 Beyond the simple pipeline

Deeper and wider machines (Chapter 9) use the same ideas:

- **Deeper pipelines**: shorter stages ⇒ higher clock, but bigger mispredict penalty.
- **Superscalar**: fetch and execute 2–8 instructions per cycle (CPI < 1).
- **Out-of-order execution**: run later independent instructions while an earlier one waits for data.

Exercises

6.1 10^{10} instructions, CPI 2, 2.5 GHz. Execution time?

6.2 Stage delays 120, 100, 180, 140, 90 ps. Pipelined clock period? Speed-up over single-cycle for a long program?

6.3 Draw the pipeline diagram for `ldr r1,[r0]; add r2,r1,r3; sub r4,r5,r6` with forwarding. How many cycles in total?

6.4 Rewrite the code in 6.3 to remove the bubble by reordering instructions.

6.5 70% of a program is parallel. What is the maximum speed-up with unlimited cores?

6.6 A loop executes a branch 100 times (taken 99, then not taken). With a 1-bit predictor that remembers the last outcome, how many mispredictions per execution of the loop (assuming it starts “taken”)? And if the loop is run again?

Chapter 7

The Memory Hierarchy: Registers, L1, L2, L3, RAM

7.1 The memory wall

A CPU can execute an instruction in ≈ 0.25 ns, but reading a value from main memory (DRAM) takes $\approx$ 60–100 ns, which is 250–400 cycles! If the CPU waited for DRAM at each load, it would be idle 99% of the time. The solution is a **hierarchy**: small, fast memories close to the CPU holding copies of what is likely to be needed soon.

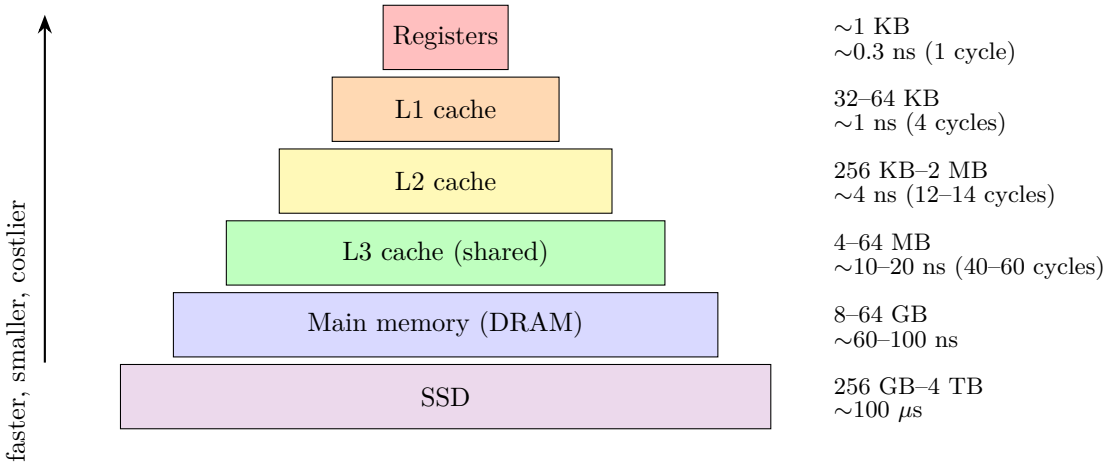


Figure 7.1: The memory hierarchy (typical figures for a modern desktop/phone-class CPU).

Analogy

You are cooking: spices on the counter (registers, 1 second away), items in the kitchen cupboard (L1), the fridge (L2), the pantry in the corridor (L3), the supermarket (DRAM), and ordering from another country (disk).

7.2 Why the hierarchy works: locality

Programs are not random. They show two kinds of **locality**:

Temporal locality A recently used item will likely be used again soon (loop counters, a function being called repeatedly).

Spatial locality Items near a recently used item will likely be used soon (consecutive array elements, sequential instructions).

Caches exploit both: they keep recently used data (temporal) and fetch whole **cache lines** of 64 bytes at a time (spatial).

Worked example 7.1: Locality in action: row vs column order

A 1024×1024 array of 4-byte integers stored in row-major order (C). A cache line is 64 bytes = 16 integers.

1. **Row by row** (`a[i][j]`, j varies fastest): consecutive elements are consecutive in memory. One miss brings in 16 integers, so 15 of the next 16 accesses hit. Miss rate $= 1/16 = 6.25\%$.
2. **Column by column** (`a[j][i]`, j varies fastest): each access jumps $1024 \times 4 = 4096$ bytes. Each access lands on a different line. Miss rate $\approx 100\%$ (for a large array).
3. Same program logic, same number of operations; the second can easily be 5–20× slower. This is one of the most important facts in practical performance work.

7.3 Memory technologies

	SRAM (caches)	DRAM (main memory)
Cell	6 transistors (two cross-coupled inverters + 2 access)	1 transistor + 1 capacitor
Stores bit as	state of a feedback loop	charge on a tiny capacitor
Speed	very fast (sub-ns to few ns)	slower (tens of ns)
Density / cost	low density, expensive	high density, cheap
Refresh?	no	yes, every ≈ 64 ms (charge leaks)
Reading	non-destructive	destructive: must be rewritten

Key idea

Speed costs area. SRAM is built from the flip-flop-like feedback ideas of Chapter 3 and can be placed on the same chip as the CPU. DRAM is a separate chip (or stacked chips) accessed over a bus; reading it takes a full "row activation" and the bandwidth is shared by all cores.

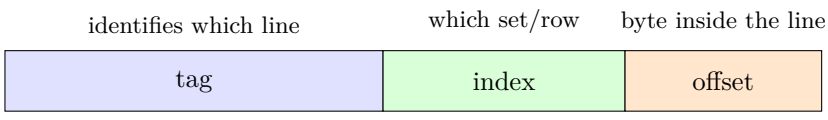
7.4 How a cache works

A cache stores **lines** (also called blocks), typically 64 bytes. Each line is stored together with a **tag** (which memory address it belongs to) and a **valid** bit. For a request:

- **Hit**: the line is in the cache: fast.
- **Miss**: ask the next level; when the line arrives, store it (maybe evicting another).

7.4.1 Splitting an address

An address is cut in three fields:



- **offset** $= \log_2$(line size) bits: selects the byte within the line.
- **index** $= \log_2$(number of sets) bits: selects where to look.
- **tag** = the remaining upper bits, compared to see if it is the right line.

7.4.2 Direct-mapped cache

Each memory line can go in exactly one place: the line number mod number of lines.

Worked example 7.2: Splitting an address (direct-mapped)

Cache: 4 KiB, direct-mapped, 16-byte lines, 32-bit addresses. Address `0x12345678`.

1. Number of lines = 4096/16 = 256 = 2^8 ⇒ index = 8 bits.
2. Line size 16 bytes = 2^4 ⇒ offset = 4 bits.
3. Tag = 32 − 8 − 4 = 20 bits.
4. `0x12345678` in binary: `0001 0010 0011 0100 0101 0110 0111 1000`.
5. Offset = last 4 bits `1000` = 8. Index = next 8 bits `0110 0111` = `0x67` = 103. Tag = top 20 bits = `0x12345`.
6. The hardware goes to line 103; if it is valid and its tag equals `0x12345`, it is a hit and byte 8 is returned.

	V	Tag	Data (64 bytes)
set 0	1	3A1	[64 bytes]
set 1	0	–	–
set 2	1	00F	[64 bytes]
set 3	1	3A1	[64 bytes]

Figure 7.2: A cache is a small table indexed by set number; each entry holds a valid bit, a tag and the data line.

7.4.3 Conflicts and associativity

Problem: two addresses that map to the same index keep evicting each other (**conflict misses**), even if the rest of the cache is empty.

Worked example 7.3: Direct-mapped vs 2-way

Tiny cache: 4 words of 4 bytes. Access byte addresses `0x00`, `0x04`, `0x10`, `0x00`.

1. **Direct-mapped** (4 lines): line number = addr/4, index = line mod 4. `0x00`→ line 0, index 0. `0x04`→ index 1. `0x10`=16→ line 4, index 0 (conflict!). `0x00` again → index 0.
2. Sequence: miss, miss, miss (evicts `0x00`), **miss** (evicts `0x10`). 0 hits.
3. **2-way set-associative** (2 sets of 2 ways): index = line mod 2. `0x00`→ set 0, `0x04`→ set 1, `0x10`→ set 0 (second way), `0x00`→ set 0: found in way 0.
4. Sequence: miss, miss, miss, **hit**.

Associativity options:

Type	Description
Direct-mapped (1-way)	one possible slot; simplest and fastest; many conflicts.
n-way set-associative	n slots per set; all tags in the set compared in parallel. Typical L1: 8-way; L2: 8–16 way.
Fully associative	any line anywhere; no conflicts but expensive (used for small structures like the TLB).

When a set is full we must choose a **victim**. The common policy is **LRU** (Least Recently Used) or a cheap approximation of it.

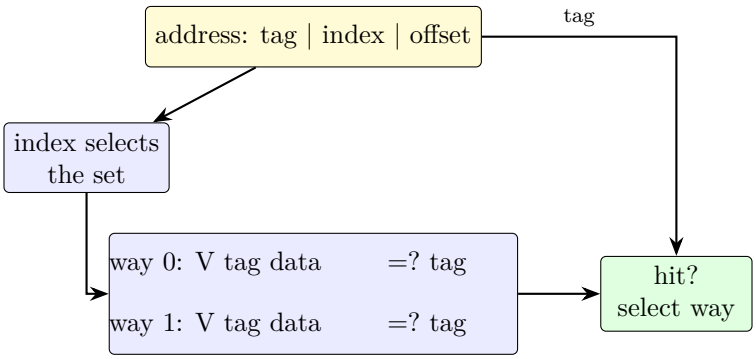


Figure 7.3: Lookup in a set-associative cache: use the index to find the set, compare the tag of every way in parallel, pick the matching way.

Worked example 7.4: Real L1 data cache geometry

32 KiB, 8-way, 64-byte lines, 32-bit addresses.

1. Number of lines $= 32768/64 = 512$.
2. Number of sets $= 512/8 = 64 = 2^6 \Rightarrow$ index $= 6$ bits.
3. Offset $= \log_2 64 = 6$ bits.
4. Tag $= 32 - 6 - 6 = 20$ bits.

A lookup compares 8 tags of 20 bits at once. This is why more ways cost more energy and time.

7.4.4 Types of misses (the three C's)

- **Compulsory**: the first access to a line (cold start).
- **Capacity**: the working set is bigger than the cache.
- **Conflict**: too many lines map to the same set.

7.4.5 Writes

Policy	Behaviour
Write-through	every store goes to the cache *and* to the next level: simple, but slow/traffic heavy.
Write-back	store only updates the cache and marks the line **dirty**; the line is written to memory when evicted. Standard in modern CPUs.
Write-allocate	on a write miss, first load the line, then write to it. Usually combined with write-back.

7.5 Measuring cache performance: AMAT

$$\text{AMAT} = t_{hit} + \text{miss rate} \times \text{miss penalty}$$

With multiple levels the miss penalty of L1 is itself the AMAT of L2 and below.

Worked example 7.5: AMAT for a three-level system

L1 hit time 4 cycles, L1 miss rate 5%. L2 hit time 14 cycles, L2 local miss rate 20%. Memory access 200 cycles.

1. Cost of going past L1 $= 14 + 0.20 \times 200 = 14 + 40 = 54$ cycles.
2. AMAT $= 4 + 0.05 \times 54 = 4 + 2.7 = 6.7$ cycles.
3. Without any cache: 200 cycles. The caches give a $\approx 30\times$ speed-up.

4. Improve L1 miss rate to 4%: AMAT $= 4 + 0.04 \times 54 = 6.16$. A 1 percentage-point improvement saves 0.54 cycles per access, which matters when there are billions of accesses.

7.6 The real L1 / L2 / L3 hierarchy

- **L1** is split into an instruction cache (L1I) and a data cache (L1D), each 32–64 KB, private to each core. It is accessed in 3–5 cycles and fits right into the pipeline's IF and MEM stages.
- **L2**: 256 KB–2 MB per core (sometimes shared between a cluster of cores), 10–15 cycles.
- **L3** (last-level cache, LLC): many MB, shared by all cores, 30–60 cycles. Also the meeting place for coherence traffic.
- **Inclusive vs exclusive**: in an inclusive hierarchy L2 contains a copy of everything in L1; in an exclusive one a line lives in only one level, saving space.
- **Prefetchers** watch your access pattern (e.g. stride +64 bytes) and fetch lines before you ask for them.
- The cache line size (64 B) is also the unit of transfer between levels and between cores.

Worked example 7.6: Where does a load go?

`ldr r0, [r1]` with r1 = `0x12345678`.

1. Check L1D: miss.
2. Check L2: miss.
3. Check L3: miss (cost so far $\approx$ 60 cycles).
4. Send request to the DRAM controller: wait $\approx$ 200 cycles.
5. The 64-byte line returns and is installed in L3, L2 and L1.
6. The word at offset `0x38`= 56 is forwarded to r0. The next load to `0x1234567C` hits in L1 in 4 cycles.

7.7 Writing cache-friendly code

1. Access arrays sequentially (in memory order).
2. Keep the *working set* small: process data in blocks (tiling) so a block fits in L1/L2.
3. Prefer contiguous arrays over linked lists: pointers scatter data across many lines.
4. Avoid power-of-two strides that map everything to the same sets.

Worked example 7.7: Tiling a matrix multiplication (idea)

Naive $C = A \times B$ with $n = 1024$ reads whole columns of B repeatedly; each is far larger than L1, so most reads miss. Instead, split the matrices into 32×32 tiles. One tile of 4-byte numbers $= 32 \cdot 32 \cdot 4 = 4$ KiB, and three tiles (A, B, C) = 12 KiB, which fits in a 32 KiB L1. While computing with these tiles every element is reused 32 times from L1 instead of fetched from memory. Optimised libraries (BLAS) do exactly this, gaining 10–100×.

Exercises

7.1 A 16 KiB direct-mapped cache with 32-byte lines and 32-bit addresses: how many index, offset and tag bits?

7.2 Split the address `0x0000A4C8` for the cache of 7.1 (give tag, index, offset in hex).

7.3 64 KiB, 4-way, 64-byte lines: number of sets, index bits, tag bits (32-bit addresses)?

7.4 L1: 3 cycles, 10% miss. L2: 12 cycles, 25% local miss. Memory: 150 cycles. Compute the AMAT.

7.5 Trace the access pattern `0x00`, `0x08`, `0x00`, `0x40`, `0x00` on a direct-mapped cache of 4 lines of 16 bytes. Which are hits?

7.6 Why is write-back better than write-through when a loop increments the same counter a million times?

7.7 Explain why traversing a linked list is usually slower than traversing an array of the same length.

Chapter 8

Virtual Memory

8.1 The problem

In Chapter 5, `ldr r0, [r1]` used r1 as a memory address. But a modern system runs hundreds of programs at once. If they all used real RAM addresses directly:

- one buggy program could overwrite another program (or the OS);
- each program would need to know which parts of RAM are free;
- programs bigger than RAM could not run.

The solution is **virtual memory**: every process sees its own private address space ("virtual addresses"), and a hardware unit, the **MMU** (Memory Management Unit), translates these into **physical addresses** in real RAM on every access.

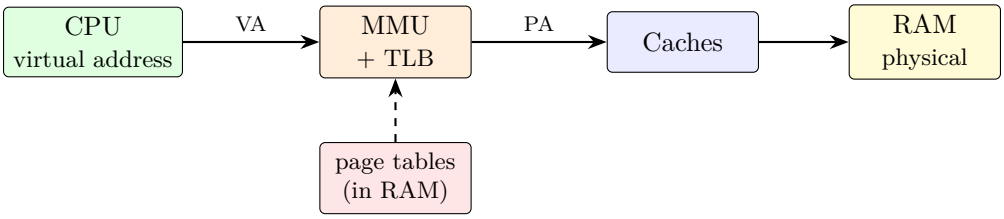
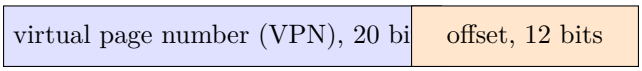


Figure 8.1: The MMU translates every address issued by the CPU.

8.2 Pages and page tables

Memory is divided into fixed-size blocks, called **pages** (virtual) and **frames** (physical). The usual size is **4 KiB** (2^{12} bytes). A virtual address is split in two:



The offset is copied unchanged. The VPN is used to look up a **page table entry (PTE)**, which contains the physical frame number plus permission bits (valid, read, write, execute, user/kernel).

Worked example 8.1: A toy translation

16-bit virtual addresses, 1 KiB pages (2^{10}), so offset = 10 bits and VPN = 6 bits. The page table of the process contains:

VPN	valid	frame
0	1	5
1	1	2
2	0	–
3	1	7

Translate virtual address `0x0C25`.

1. Binary: `0000 1100 0010 0101`.
2. VPN = top 6 bits `000011` = 3. Offset = low 10 bits `00 0010 0101` = `0x025`.
3. Page table entry 3: valid, frame 7.
4. Physical address = $7 \times 1024 + \texttt{0x025} = \texttt{0x1C00} + \texttt{0x025} = \texttt{0x1C25}$.

Now translate `0x0825`: VPN = 2, entry invalid ⇒ **page fault**: the CPU traps to the operating system.

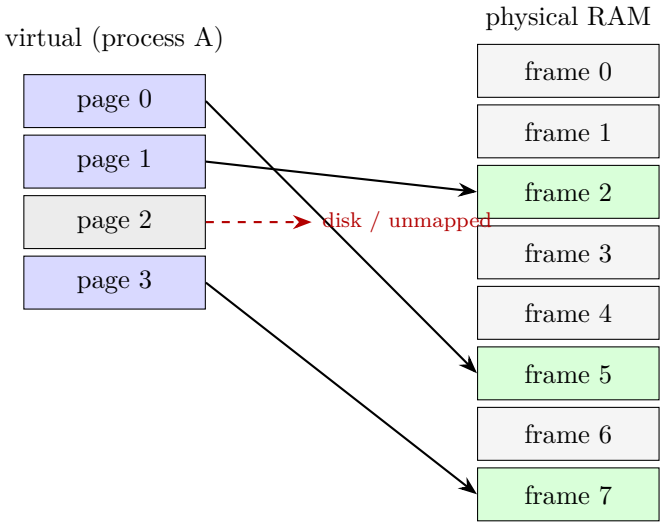


Figure 8.2: Pages of a process can be anywhere in physical memory, in any order. Process A cannot even name the frames of process B.

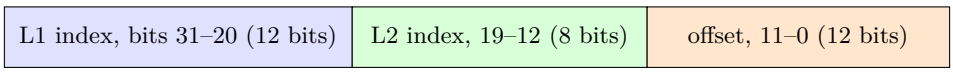
Key idea

Virtual memory gives **isolation** (each process has its own map), **simplicity** (every program thinks it owns addresses `0x00000000`–`0xFFFFFFFF`), and **flexibility** (pages can be swapped to disk, shared between processes, or mapped to files).

8.3 Page-table size and multi-level tables

A flat table for a 32-bit address space with 4 KiB pages needs $2^{20} = 1\,048\,576$ entries $\times 4$ bytes = 4 MiB *per process*, mostly empty. The fix: a **two-level** table that only allocates the pieces actually used.

ARMv7 (short-descriptor format) splits the address as:



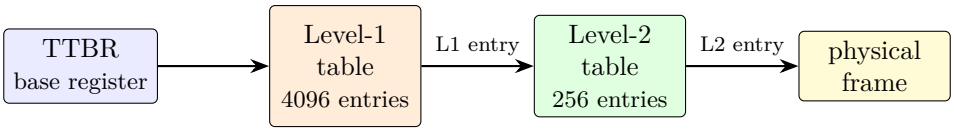


Figure 8.3: Two-level page walk: two table lookups, then the data access itself.

Worked example 8.2: Splitting an ARMv7 virtual address

Virtual address `0x00403ABC`.

1. Bits 31–20: `0x004` ⇒ L1 index = 4.
2. Bits 19–12: `0x03` ⇒ L2 index = 3.
3. Bits 11–0: `0xABC` ⇒ offset.
4. Walk: read L1 table entry 4 (1 memory access) → gives the address of an L2 table; read its entry 3 (2nd access) → gives the physical frame; add offset `0xABC`; finally access the data (3rd access).

So a naive translation turns *one* load into *three* memory accesses. We need something to avoid this.

8.4 The TLB: a cache for translations

The **Translation Lookaside Buffer (TLB)** is a small, very fast cache (typically 32–512 entries, often fully associative) that remembers recent VPN → frame translations. Hit: translation in about 1 cycle, in parallel with the L1 access. Miss: the hardware (or OS) walks the page tables.

Worked example 8.3: Effective memory access time with a TLB

TLB lookup 1 ns, memory access 100 ns, TLB hit rate 99%, two-level walk.

1. TLB hit: $1 + 100 = 101$ ns.
2. TLB miss: $1 + 100 + 100 + 100 = 301$ ns (two table reads and the data).
3. EAT $= 0.99 \times 101 + 0.01 \times 301 = 99.99 + 3.01 = 103.0$ ns.

Only 3% slower than the ideal 100 ns. Without a TLB every access would cost 300 ns.
TLB reach: 64 entries $\times 4$ KiB = 256 KiB. Programs touching much more memory randomly will suffer TLB misses. That is why operating systems offer *huge pages* (2 MiB), giving 512× more reach per entry.

8.5 Page faults: using the disk as extra memory

If a page is not in RAM (valid bit = 0), the CPU raises a **page fault** exception. The operating system:

1. finds the page on disk (or creates a fresh zeroed page);
2. picks a free frame, or evicts another page (writing it to disk if dirty);
3. loads the page, updates the page table and TLB;
4. restarts the instruction that faulted.

Worked example 8.4: Why page faults must be rare

Memory access 100 ns, page-fault service 8 ms = 8 000 000 ns, fault probability p.

1. EAT $= (1 - p) \cdot 100 + p \cdot 8\,000\,000 \approx 100 + 7\,999\,900\,p$ ns.
2. To keep the slowdown below 10%, i.e. EAT ≤ 110: $7\,999\,900\,p \leq 10 \Rightarrow p \leq 1.25 \times 10^{-6}$.
3. That is at most one fault every 800 000 accesses. This is why a computer that starts *swapping* (thrashing) feels frozen.

8.6 Memory protection and privilege

PTE permission bits are checked on each access:

- **Read/Write/Execute**: code pages are read+execute (not writable), data pages are read+write (not executable). This stops many attacks.
- **User/Kernel**: user programs cannot access kernel pages. A violation raises a fault ("segmentation fault").

Entering the kernel happens with `svc` (Chapter 5) or when an interrupt occurs; the CPU switches mode, and the kernel switches page tables when it changes process (a *context switch*), which also means the TLB must be flushed or tagged (ASID/PCID).

8.7 Caches and virtual addresses

Should the L1 cache be indexed by virtual or physical addresses? Many L1 caches are **VIPT**: Virtually Indexed, Physically Tagged. The index bits come directly from the page offset (identical in virtual and physical addresses), so the cache set can be read at the same time as the TLB translates the upper bits. The tag compare then uses the physical address. This is also why L1 size per way is often limited to the page size: 64 sets $\times$ 64 B = 4 KiB (one page), so a 32 KiB L1 needs 8 ways.

Exercises

8.1 Pages are 4 KiB. How many offset bits? How many pages in a 32-bit space?

8.2 Use the toy page table of the chapter to translate `0x0425` and `0x01FF`.

8.3 Split the ARMv7 address `0x80123456` into L1 index, L2 index and offset.

8.4 TLB hit rate 98%, TLB 1 ns, memory 80 ns, one-level page table. Compute the EAT.

8.5 Why does the virtual memory system make it impossible for one process to read another's data (unless the OS allows it)?

8.6 A TLB has 128 entries and 4 KiB pages: what is its reach? And with 2 MiB pages?

Chapter 9

Inside a Modern CPU: Parallelism at Every Level

Clock speed stopped growing around 2005 (power wall, Chapter 3), so designers found speed through **parallelism**. A modern core exploits parallelism at four levels:

Level	Idea	Hardware
Instruction-level (ILP)	run several instructions per cycle	superscalar, out-of-order
Data-level (DLP)	one instruction on many data	SIMD / NEON, GPU
Thread-level (TLP)	several instruction streams	SMT, multicore
Request-level	many machines	clusters, cloud

9.1 Superscalar and out-of-order execution

A **superscalar** core has several execution units (ALUs, load/store units, FPU) and can issue 2–8 instructions per cycle if they are independent. **Out-of-order (OoO)** execution lets an instruction run as soon as its inputs are ready, rather than strictly in program order.

Worked example 9.1: Why out-of-order helps

```
1    ldr  r1, [r0]        @ (1) cache miss: result in ~200 cycles
2    add  r2, r1, #1      @ (2) needs r1
3    mov  r3, #5          @ (3) independent
4    add  r4, r3, r3      @ (4) independent of (1) and (2)
```

1. **In-order**: (2) blocks waiting for r1; (3) and (4) are stuck behind it. Total ≈ 200+ cycles with idle hardware.
2. **Out-of-order**: (1) starts, (2) waits in a buffer, (3) and (4) execute immediately. When the data arrives, (2) runs. The work of (3) and (4) was hidden inside the wait.
3. The results are still *committed* (made visible) in program order using a **reorder buffer (ROB)**, so the program behaves as if it ran in order.

Key mechanisms:

Register renaming The CPU has more physical registers (100–300) than the 16 architectural ones. Renaming removes false dependencies (two instructions reusing `r1` for unrelated values).

Speculation The CPU executes beyond unresolved branches, guided by the branch predictor (Chapter 6); wrong-path work is discarded.

Reorder buffer Keeps hundreds of instructions in flight and retires them in order.

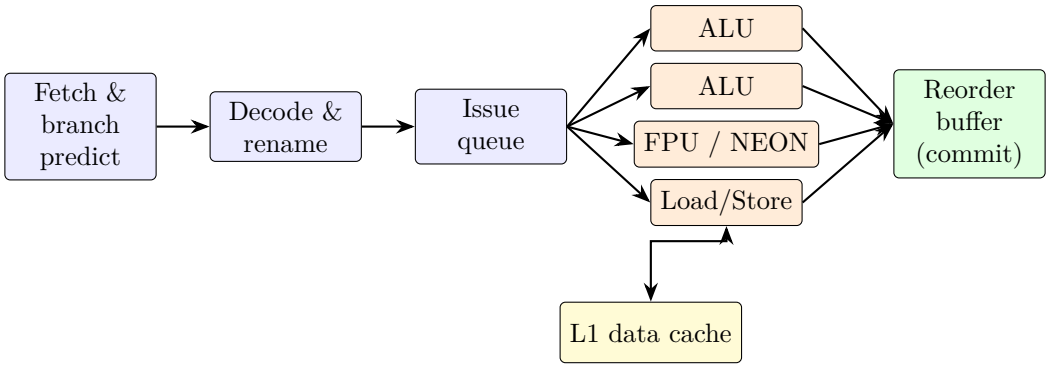
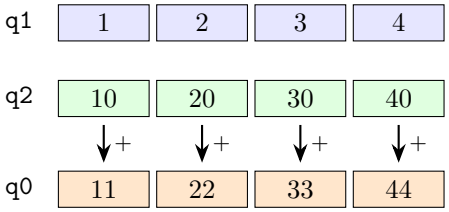


Figure 9.1: Simplified out-of-order superscalar core: instructions are fetched in order, executed out of order and committed in order.

9.2 SIMD: one instruction, many data

SIMD (Single Instruction Multiple Data) applies one operation to a whole vector held in a wide register. On ARM this is **NEON**: 128-bit registers `q0`–`q15`, i.e. 4 floats, 8 halfwords or 16 bytes.




```
    vld1.32 {q1}, [r0]!      @ load 4 floats from a[], advance r0 by 16
    vld1.32 {q2}, [r1]!      @ load 4 floats from b[]
    vadd.f32 q0, q1, q2      @ 4 additions in a single instruction
    vst1.32 {q0}, [r2]!      @ store 4 results into c[]
```

Worked example 9.2: SIMD speed-up

Adding two arrays of 1 000 000 floats.

1. Scalar loop: 1 000 000 additions $\Rightarrow 10^6$ `vadd`/`fadd` instructions (plus loads/stores).
2. NEON: each `vadd.f32` does 4 additions $\Rightarrow$ 250 000 vector instructions.
3. Ideal speed-up 4×. In practice memory bandwidth often limits it (Chapter 10), but compute-heavy kernels reach 3–4×.

Compilers auto-vectorise simple loops (`gcc -O3`), and x86 has the same idea with SSE/AVX (128–512 bits).

9.3 Multicore and multithreading

Instead of one big core, put 4, 8 or 64 cores on a chip. Each core has private L1 (and often L2); they share L3 and memory.

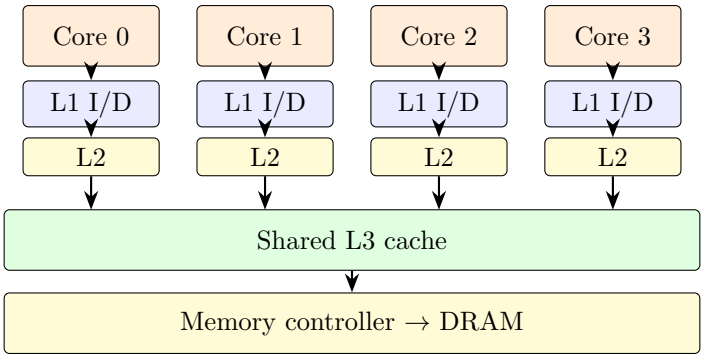


Figure 9.2: A quad-core processor: private L1/L2, shared L3.

SMT (hyper-threading). One core keeps two thread contexts and fills idle execution slots from the other thread when one stalls (e.g. on a cache miss). It costs little hardware and gives 10–30% more throughput.

big.LITTLE. Many ARM chips mix fast power-hungry cores with slow efficient ones; the OS moves work to the best core for the job.

9.4 Cache coherence

With several private caches there is a new problem: core A and core B may both hold a copy of address x. If A writes x, B must not keep reading the stale copy. Hardware keeps the copies **coherent** using a protocol such as **MESI**. Each cache line is in one of four states:

State	Meaning	Memory copy
Modified	only this cache has it, and it has been changed	stale
Exclusive	only this cache has it, unchanged	up to date
Shared	several caches may have it, unchanged	up to date
Invalid	not usable	–

Worked example 9.3: MESI in action

Variable x is in memory. Follow the state of its line in core A and core B.

Event	A	B	What happens
start	I	I	
A reads x	E	I	loaded from memory; nobody else has it
B reads x	S	S	both share the clean line
A writes x	M	I	A sends an *invalidate*; B's copy dies
B reads x	S	S	A supplies the data (and writes back); both shared

Careful!

False sharing. Two threads update *different* variables that happen to live in the *same* 64-byte line (e.g. `counter[0]` and `counter[1]`). Each write invalidates the other core's copy, so the line ping-pongs between cores and the program can run many times slower than a single thread. Fix: pad each counter to its own cache line.

9.5 Memory ordering and synchronisation

Because of store buffers and out-of-order execution, other cores may see writes in a different order than the program wrote them. ARM has a *weak memory model*: you need explicit barriers (`dmb`) or atomic instructions (`ldrex`/`strex`) to build locks. In high-level languages, use mutexes and atomics; they insert the right instructions.

Worked example 9.4: An atomic increment on ARMv7

```
retry:  ldrex   r1, [r0]        @ load counter and mark address as monitored
        add     r1, r1, #1
        strex   r2, r1, [r0]    @ store only if nobody touched it; r2=0 on success
        cmp     r2, #0
        bne     retry           @ someone interfered: try again
```

Exercises

9.1 Which instructions in `ldr r1,[r0]; add r2,r1,#1; mov r3,#7; mul r4,r3,r3; sub r5,r2,r4` can start before the load returns?

9.2 A NEON register holds 16 bytes. How many 8-bit pixels can `vadd.u8` process per instruction? How many instructions to brighten a 1920×1080 greyscale image?

9.3 Complete the MESI table: A reads x, B writes x, A reads x.

9.4 Explain why using one shared counter with a lock for 8 threads may scale worse than 8 per-thread counters summed at the end.

9.5 Why does an out-of-order CPU need a reorder buffer?

Chapter 10

The GPU: Massive Parallelism

10.1 Two design philosophies

A CPU is built to make *one thread* finish as fast as possible (low **latency**): big caches, branch predictors, out-of-order logic. A GPU is built to finish *as much total work as possible per second* (high **throughput**): thousands of simple lanes and very little control logic per lane.

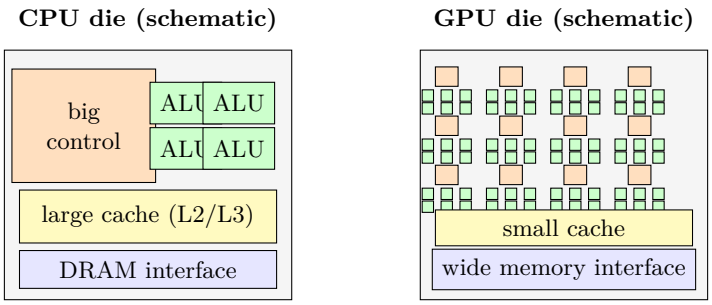


Figure 10.1: Same silicon area, different priorities: the CPU spends transistors on control and caches; the GPU spends them on arithmetic units.

Analogy

A CPU is a handful of brilliant chefs who can cook any complicated dish quickly. A GPU is a stadium kitchen with 10 000 cooks who can each only follow a simple recipe, but all cook at once. For one fancy dish, take the CPU. To make 10 000 identical sandwiches, take the GPU.

10.2 The SIMT execution model

GPUs use **SIMT** (Single Instruction, Multiple Threads). The programmer writes code for *one thread*; the hardware launches thousands of copies, each working on a different data element.

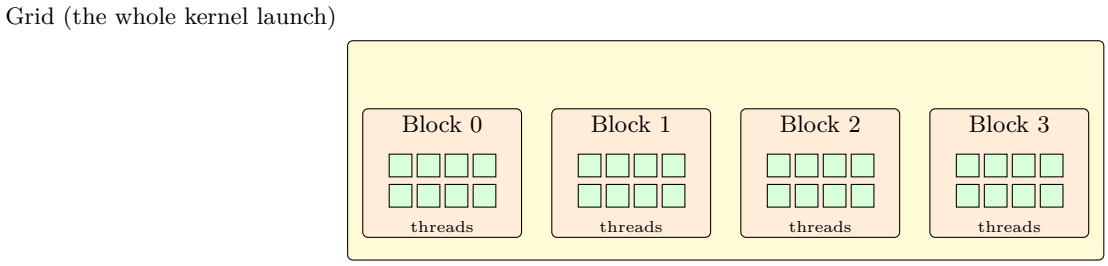


Figure 10.2: Grid → blocks → threads. A block runs on one Streaming Multiprocessor (SM); blocks are independent of each other.

Hardware terms (NVIDIA names; AMD and ARM Mali have equivalents):

Thread one lane running the kernel for one element.

Warp a group of 32 threads executing the same instruction in lock-step (the real unit of execution).

Block up to 1024 threads that can cooperate through fast *shared memory* and synchronise with a barrier.

SM a Streaming Multiprocessor: a mini-processor with many lanes, a register file, shared memory and schedulers. A GPU has dozens of them.

10.2.1 Your first kernel: vector addition

```
__global__ void vadd(const float *a, const float *b, float *c, int n)
    {
    int i = blockIdx.x * blockDim.x + threadIdx.x;    // my global
    index
    if (i < n)                                        // guard the
    tail
        c[i] = a[i] + b[i];
}

// host code
int threads = 256;
int blocks  = (n + threads - 1) / threads;            // round up
vadd<<<blocks, threads>>>(a, b, c, n);
```

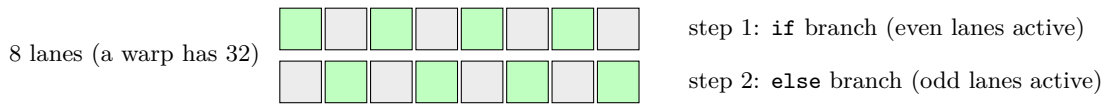
Worked example 10.1: Thread indexing

$n = 1\,000\,000$ elements, 256 threads per block.

1. Blocks $= \lceil 1\,000\,000/256 \rceil = \lceil 3906.25 \rceil = 3907$.
2. Total threads launched $= 3907 \times 256 = 1\,000\,192$; the last 192 threads fail the `i < n` test and do nothing.
3. The thread with `blockIdx.x`$= 2$, `threadIdx.x`$= 5$ computes $i = 2 \times 256 + 5 = 517$, so it does `c[517] = a[517] + b[517]`.
4. These 256 threads form $256/32 = 8$ warps per block.

10.3 Warp divergence

All 32 threads of a warp share one instruction pointer. If they take different sides of an `if`, the hardware executes *both* paths one after another, masking off the lanes that are not active.



Worked example 10.2: Cost of divergence

`if (i % 2 == 0) A(); else B();` where A and B each take 100 cycles.

1. Within one warp, half the lanes go each way, so the warp executes A (100 cycles) then B (100 cycles) = 200 cycles.
2. If the condition were uniform across the warp (e.g. `blockIdx.x % 2`), each warp runs only one path: 100 cycles.

Divergence can halve (or worse) the throughput. Write kernels so that neighbouring threads follow the same path.

10.4 GPU memory hierarchy

Level	Scope	Typical size	Speed
Registers	per thread	up to 255 per thread (256 KB per SM)	1 cycle
Shared memory	per block (on-chip SRAM)	48–228 KB per SM	∼20–30 cycles
L1 cache	per SM	combined with shared memory	∼30 cycles
L2 cache	whole GPU	4–96 MB	∼200 cycles
Global memory (VRAM)	whole GPU	8–192 GB (GDDR / HBM)	∼400–800 cycles

Bandwidth is the strength: GDDR6X gives ≈ 1 TB/s and HBM3 several TB/s, compared with 50–100 GB/s for a CPU's DRAM.

10.4.1 Coalescing

Global memory is read in aligned chunks of 32–128 bytes. If the 32 threads of a warp read 32 consecutive floats ($32 \times 4 = 128$ bytes), hardware merges them into **one** transaction. If they read with a stride of 32 floats, it needs **32** transactions: 32 times the memory traffic for the same useful data. This is the GPU counterpart of spatial locality in CPU caches (Chapter 7).

10.4.2 Hiding latency with many threads

A CPU hides memory latency with caches and out-of-order execution. A GPU hides it by *switching to another warp*. While warp 1 waits 500 cycles for data, the scheduler runs warps 2, 3, 4... at zero switching cost (all warps keep their registers on-chip). So the GPU needs many resident warps (high **occupancy**) to stay busy.

10.5 How fast, really? The roofline model

Worked example 10.3: Peak compute of an imaginary GPU

80 SMs, each with 128 FP32 lanes, clock 1.5 GHz, each lane can do one fused multiply-add (2 FLOPs) per cycle.

1. Lanes $= 80 \times 128 = 10\,240$.
2. FLOPs per cycle $= 10\,240 \times 2 = 20\,480$.
3. Peak $= 20\,480 \times 1.5 \times 10^9 \approx 30.7$ TFLOP/s.

But peak is only reachable if data arrives fast enough. Define the **arithmetic intensity**: floating-point operations per byte moved from memory. Performance $\leq \min(\text{peak}, \text{bandwidth} \times \text{intensity})$.

Worked example 10.4: Vector add is memory-bound

Per element: 1 addition, reads 2 floats and writes 1 float = 12 bytes.

1. Intensity $= 1/12 \approx 0.083$ FLOP/byte.
2. With 1 TB/s of bandwidth: $10^{12} \times 0.083 \approx 83$ GFLOP/s.
3. That is only 0.27% of the 30.7 TFLOP/s peak. The ALUs are idle; the memory system is the bottleneck.

In contrast, a tiled matrix multiplication reuses each loaded value many times in shared memory and can reach intensities above 50 FLOP/byte, so it can approach peak. This is why GPUs shine in deep learning (which is dominated by matrix multiplication), and why modern GPUs even add **tensor cores** that do small matrix multiply-adds in one instruction.

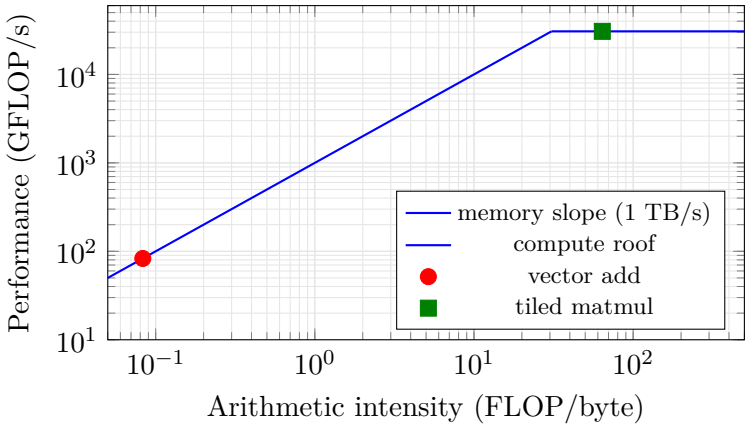


Figure 10.3: Roofline of the example GPU. Vector add sits on the sloped (memory-bound) part; a well-tiled matrix multiply reaches the flat compute roof.

10.6 Talking to the GPU

- A discrete GPU has its own memory (VRAM) and sits on the PCIe bus. Data must be copied across before and after the kernel.

Worked example 10.5: Is the copy worth it?

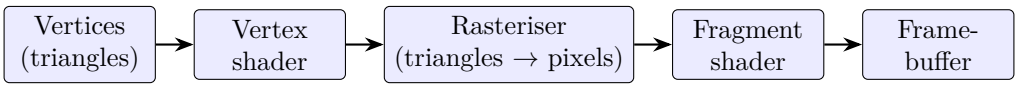
Move 1 GB to the GPU over PCIe at 16 GB/s.

1. Copy time = 1/16 s = 62.5 ms (and the same back).
2. If the kernel takes only 2 ms on the GPU, the transfer dominates: total ≈ 127 ms. A CPU doing the vector add in 20 ms would win.
3. GPUs pay off when the computation per byte transferred is high, or when data stays on the GPU between kernels.

Phones and laptops often use **unified memory**: the CPU and GPU share the same DRAM, removing the copy.

10.7 The graphics pipeline in one picture

GPUs were invented for graphics. Even for general computing it helps to know the original job:



A 4K frame has $3840 \times 2160 \approx 8.3$ million pixels; at 60 frames per second the fragment shader runs about 5×10^8 times per second, perfectly parallel work. The same hardware, programmed with CUDA, OpenCL, Vulkan compute or Metal, became the engine of scientific computing and AI.

10.8 CPU vs GPU: when to use which

	CPU	GPU
Cores	4–64 complex	thousands of simple lanes
Strength	latency, branching, serial code	throughput, regular data parallelism
Memory bandwidth	50–100 GB/s	0.5–5 TB/s
Good for	OS, databases, compilers, control logic	graphics, matrix maths, image/video, ML training
Bad for	massively parallel number crunching	irregular branching, tiny tasks

Exercises

10.1 $n = 100\,000$ elements, 128 threads per block. How many blocks? How many threads are idle because of the guard?

10.2 Which global index does thread 7 of block 40 compute (256 threads per block)?

10.3 A warp executes a three-way `if/else if/else` where each lane takes a different path for 3 groups of lanes. What is the worst-case slow-down?

10.4 Matrix transpose reads rows but writes columns of a 4096×4096 float matrix. Explain with coalescing why one side is slow.

10.5 Compute the arithmetic intensity of $y[i] = a \cdot x[i] + y[i]$ (SAXPY) with 4-byte floats (2 FLOPs per element). Is it memory- or compute-bound on the example GPU?

10.6 List three reasons a GPU needs thousands of threads in flight to be efficient.

Chapter 11

I/O, Interrupts and the Whole Machine

11.1 Talking to devices

A CPU that could only compute would be useless. It needs to read keys, draw pixels, store files and use the network. Devices are reached through registers mapped into the address space: **memory-mapped I/O**. A load or store to a special address does not go to RAM but to a device.

Worked example 11.1: Sending a character to a serial port (UART)

Assume the UART data register is at `0x101F1000` (as on QEMU's *versatilepb* board) and r1 holds the character to send.

```
    ldr   r0, =0x101F1000      @ address of the UART data register
    strb  r1, [r0]             @ writing a byte there transmits it
```

This looks like an ordinary store, but the bus routes it to the UART, which shifts the bits out on a wire. Caches must be disabled for such regions (otherwise the write would be absorbed by the cache and never reach the device).

11.1.1 Three ways to wait for a device

Method	How	Cost
Polling	CPU repeatedly reads a status register in a loop.	wastes CPU time, simple, low latency.
Interrupts	Device raises a signal; the CPU pauses its program, runs a handler, then resumes.	small overhead per event, no waiting.
DMA	A DMA engine copies blocks between device and RAM without the CPU; it interrupts at the end.	best for big transfers.

11.2 Interrupts and exceptions

An **exception** is any event that diverts the CPU from the normal flow. ARMv7 defines a vector table at fixed addresses, one slot for each type:

Exception	Vector offset	Cause
Reset	`0x00`	power-on
Undefined instruction	`0x04`	CPU cannot decode the word
Supervisor call (SVC)	`0x08`	program asked the OS for a service
Prefetch abort	`0x0C`	instruction fetch failed (e.g. page fault)
Data abort	`0x10`	load/store failed (page fault, bad address)
IRQ	`0x18`	normal hardware interrupt
FIQ	`0x1C`	fast interrupt

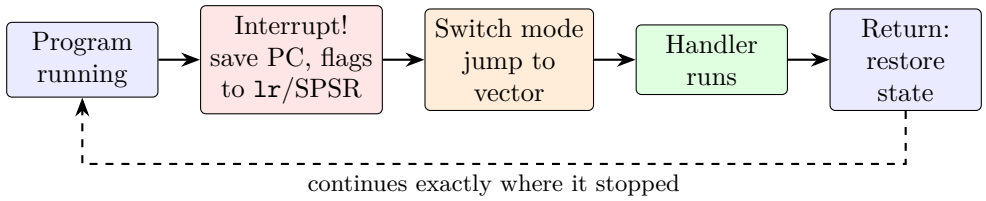


Figure 11.1: What happens on an interrupt.

Worked example 11.2: Polling versus interrupts

A network card delivers 10 000 packets per second. Handling one packet takes 5 μs of CPU work. An interrupt costs 2 μs of overhead (context save, restore, cache disruption).

1. Interrupt per packet: $10\,000 \times (5+2)$ μs $= 70$ ms of CPU every second: 7% of one core.
2. At 1 000 000 packets per second: $1\,000\,000 \times 7$ μs $= 7$ s per second, which is impossible (700%). The CPU drowns in interrupts.
3. Fix: **interrupt coalescing** or switching to polling at high load (what Linux's NAPI does): handle many packets per interrupt.

11.3 Buses and bandwidth

Link	Typical bandwidth	Used for
DDR5-6400, 1 channel (64-bit)	$6400 \times 10^6 \times 8$ B $= 51.2$ GB/s	main memory
Dual-channel DDR5-6400	102.4 GB/s	desktop RAM
PCIe 4.0 $\times$16	≈ 32 GB/s each way	GPU, fast SSD
NVMe SSD	3–7 GB/s	storage
USB 3.2 (10 Gb/s)	1.2 GB/s	peripherals
Gigabit Ethernet	0.125 GB/s	network

Note the gap: RAM is ≈ 3 times PCIe, and 800 times a gigabit network link.

11.4 From power button to login prompt

1. The CPU resets: PC points to the boot ROM (burnt into the chip).
2. The ROM initialises the clock and DRAM and loads a **bootloader** from flash or SD card.
3. The bootloader (U-Boot, UEFI) loads the **kernel** into RAM and jumps to it.
4. The kernel sets up the MMU and page tables (Chapter 8), interrupts, drivers and the first process `init`.
5. `init` starts services and finally the login screen.

11.5 The grand tour: one load, from transistor to DRAM

Let us follow the single instruction `ldr r0, [r1]` through everything we learned.

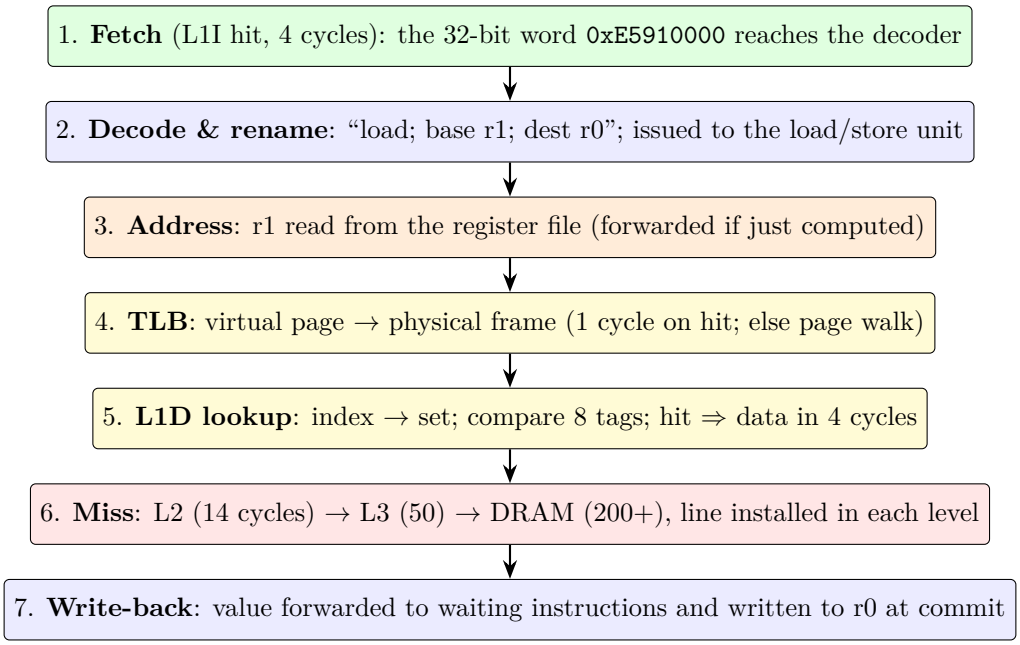


Figure 11.2: The life of a load instruction.

Underneath, each step is a few thousand transistors switching: adders (Chapter 2) built into an ALU and address generator, flip-flops (Chapter 3) in every pipeline register, SRAM cells in the caches (Chapter 7), all paced by a clock of a few hundred picoseconds.

11.6 Numbers every engineer should know

Operation	Approx. time
One clock cycle at 4 GHz	0.25 ns
Register / ALU operation	1 cycle
Branch mispredict	∼15 cycles (≈ 4 ns)
L1 cache hit	∼1 ns
L2 cache hit	∼4 ns
L3 cache hit	∼15 ns
DRAM access	∼80 ns
NVMe SSD random read	∼100 μs
Network round trip in a data centre	∼500 μs
Hard disk seek	∼5–10 ms
Round trip internet (Paris–New York)	∼80 ms

If one cycle were 1 second, an L1 hit would take 4 seconds, DRAM about 5 minutes, an SSD read about 5 days, and a disk seek about 4 months. The hierarchy exists to hide those gaps.

11.7 Where to go next

- *Digital Design and Computer Architecture: ARM Edition* (Harris & Harris): gates to a working ARM processor.

- *Computer Organization and Design: ARM Edition* (Patterson & Hennessy): the standard textbook, with pipelines, caches and parallelism.
- *Computer Systems: A Programmer's Perspective* (Bryant & O'Hallaron): what programmers must know about hardware.
- *Computer Architecture: A Quantitative Approach* (Hennessy & Patterson): the advanced reference.
- Practice: write ARM assembly with QEMU (`qemu-arm`) or a Raspberry Pi; read compiler output on godbolt.org; simulate caches with `valgrind --tool=cachegrind`; measure real latencies with `perf stat`.

Exercises

11.1 Why must a memory-mapped device register not be cached?

11.2 An interrupt handler takes 3 μs plus 1 μs overhead. What is the maximum interrupt rate a core can sustain at 50% CPU load?

11.3 How long does DMA need to move 4 MiB from a device over a 1 GB/s link? Why is it better than the CPU copying word by word?

11.4 Describe step by step what happens (in hardware and OS) when a program executes `svc #0` to write to the screen.

11.5 Starting from your answer to Chapter 3 about the clock, explain why DRAM is ≈ 300 cycles away from the CPU.

Appendix A

ARMv7 Cheat Sheet

Registers

`r0-r3`: arguments/results (caller-saved) `r4-r11`: callee-saved `r12`: scratch `r13=sp` `r14=lr` `r15=pc`

Common instructions

Instruction	Effect
`mov rd, op2`	$rd \leftarrow op2$
`mvn rd, op2`	$rd \leftarrow \sim op2$
`add / sub rd, rn, op2`	$rd \leftarrow rn \pm op2$
`rsb rd, rn, op2`	$rd \leftarrow op2 - rn$
`mul rd, rm, rs`	$rd \leftarrow rm \cdot rs$ (low 32 bits)
`and / orr / eor / bic`	bitwise AND / OR / XOR / AND-NOT
`lsl / lsr / asr / ror`	shift or rotate
`cmp rn, op2`	flags from $rn - op2$
`tst rn, op2`	flags from rn & $op2$
`ldr / str rd, [addr]`	load / store a word
`ldrb / strb, ldrh / strh`	byte / halfword
`push / pop {list}`	stack operations (full descending)
`b / bl label`	branch / branch with link
`bx lr`	return
`movw / movt rd, #imm16`	write low / high half of a register
`svc #0`	system call (number in r7 on Linux)
`ldrex / strex`	atomic read-modify-write
`dmb`	data memory barrier
`vadd.f32 q0,q1,q2`	NEON: 4 float additions

Suffixes

`S` = set flags (`adds`, `subs`). Condition codes: `eq ne cs/hs cc/lo mi pl vs vc hi ls ge lt gt le al`.

Addressing modes

`[rn]`, `[rn, #off]`, `[rn, #off]!` (pre-index), `[rn], #off` (post-index), `[rn, rm, lsl #k]`.

Memory sizes and bit-field recipes

- Cache: offset $= \log_2(\text{line})$; sets $= \text{size}/(\text{line} \times \text{ways})$; index $= \log_2(\text{sets})$; tag $=$ addr bits $-$ index $-$ offset.
- Paging: offset $= \log_2(\text{page size})$; VPN $=$ addr bits $-$ offset.
- AMAT $= t_{hit} +$ miss rate $\times$ miss penalty.
- CPU time $=$ IC $\times$ CPI $\times T_{clk}$.
- $T_{clk} \geq t_{cq} + t_{logic} + t_{setup}$.

Appendix B

Short Answers to Exercises

Chapter 1. (1.1) 202 = `0xCA`. (1.2) `11001000` = `0xC8`. (1.3) −1 = `11111111` (`0xFF`); −128 = `10000000` (`0x80`). (1.4) 13 + (−20): `00001101`+`11101100`=`11111001`= −7. (1.5) `0xD4`, `0xC3`, `0xB2`, `0xA1`. (1.6) $2^{16} = 65\,536$ bytes (64 KiB); 2^{36} bytes = 64 GiB.

Chapter 2. (2.1) Output 0,1,1,0: XOR. (2.2) $\overline{A + \overline{B}} = \overline{A} \cdot B$. (2.3) $A + \overline{A}B = A + B$. (2.4) 11 + 6 = 17: result `0001`, carry-out 1, meaning the unsigned sum does not fit in 4 bits. (2.5) $32 \times 28 = 896$. (2.6) Two muxes select between (D_0, D_1) and (D_2, D_3) using S_0; a third mux chooses between their outputs using S_1.

Chapter 3. (3.1) $T = 1/4.2\,\text{GHz} \approx 238$ ps. (3.2) $T = 400$ ps, $f = 2.5$ GHz. (3.3) Stages of 170 ps: $T = 230$ ps, $f \approx 4.35$ GHz; latency $2 \times 230 = 460$ ps; throughput speed-up $400/230 \approx 1.74$. (3.4) $P \propto 0.9^2 \times 0.9 = 0.729$, so ≈ 43.7 W. (3.5) $D_0 = \overline{Q_0}$, $D_1 = Q_1 \oplus Q_0$, $D_2 = Q_2 \oplus (Q_1 Q_0)$. (3.6) An SR latch has no clock: its output may change at any moment (not only at a controlled edge), and the input 11 is forbidden; synchronous design needs edge-triggered storage.

Chapter 4. (4.1) `0xE0454006`. (4.2) `ADD r2, r3, #1`. (4.3) Because only LDR/STR access memory; all other instructions work on registers, so the decoder has few, regular formats. (4.4) `0x900C`. (4.5) MemToReg. (4.6) ALU instructions and branches skip the memory stage; in the single-cycle design with the 800 ps clock an ADD (600 ps) wastes about 200 ps (25%).

Chapter 5. (5.1) r0 = `0x80000000`; N=1, Z=0, C=0, V=1. (5.2) `add r1, r0, r0, lsl #2` then `lsl r1, r1, #1` (that is $5x \times 2 = 10x$). (5.3) `cmp r0,#0; rsblt r0,r0,#0; bx lr`. (5.4) `mov r1,#0; mov r2,#10` / `loop: str r1,[r0],#4; subs r2,r2,#1; bne loop`. (5.5) 4! = 24; four frames of 8 bytes: 32 bytes. (5.6) `bl` overwrites `lr`, so the function's own return address would be lost. (5.7) Pre-index updates r1 first and loads from the new address; post-index loads from the old address and then updates r1.

Chapter 6. (6.1) $10^{10} \times 2/2.5 \times 10^9 = 8$ s. (6.2) Period 180 ps; single-cycle 630 ps; speed-up 3.5. (6.3) ldr: IF1 ID2 EX3 MEM4 WB5; add stalls one cycle (EX in cycle 5); sub finishes in cycle 8: 8 cycles total. (6.4) Move `sub` between `ldr` and `add`: no bubble, 7 cycles. (6.5) 1/0.3 = 3.33. (6.6) First run: 1 misprediction (at the exit). In steady state a 1-bit predictor mispredicts twice per run (the exit, then the first iteration of the next run).

Chapter 7. (7.1) 512 lines: index 9, offset 5, tag 18. (7.2) tag `0x2`, index `0x126`, offset `0x08`. (7.3) 1024 lines, 256 sets: index 8, offset 6, tag 18. (7.4) $3 + 0.1\,(12 + 0.25 \times 150) = 7.95$ cycles. (7.5) miss, hit, hit, miss (`0x40` maps to the same index as `0x00`), miss. (7.6) With write-back the counter stays in L1 as a dirty line and is written back once; write-through sends every update to memory. (7.7) List nodes are scattered, so almost every node is a new cache line, and each load depends on the previous one (no prefetching or overlap); arrays are contiguous: 16 ints per line and easy to prefetch.

Chapter 8. (8.1) 12 bits; 2^{20} pages. (8.2) `0x0425`→`0x0825`; `0x01FF`→`0x15FF`. (8.3) L1 index `0x801`, L2 index `0x23`, offset `0x456`. (8.4) Hit 81 ns, miss 161 ns; EAT $= 0.98 \times 81 + 0.02 \times 161 = 82.6$ ns. (8.5) Each process has its own page table; the MMU only reaches frames present in it, and permission bits are checked. (8.6) 128×4 KiB $= 512$ KiB; 128×2 MiB $= 256$ MiB.

Chapter 9. (9.1) `mov r3` and `mul r4` can run early; `add r2` and `sub r5` wait. (9.2) 16 pixels per instruction; $2\,073\,600/16 = 129\,600$ instructions. (9.3) A reads: A=E, B=I; B writes: B=M, A=I; A reads: both S (B supplies the data and writes back). (9.4) Lock contention serialises the threads and the lock's cache line bounces between cores; per-thread counters avoid shared writes. (9.5) To retire instructions in program order, which gives precise exceptions and lets speculation be undone.

Chapter 10. (10.1) 782 blocks; $782 \times 128 - 100\,000 = 96$ idle threads. (10.2) $40 \times 256 + 7 = 10\,247$. (10.3) Up to 3× slower (three paths executed in turn). (10.4) The column-wise side makes the 32 threads of a warp touch addresses 16 KiB apart: 32 transactions instead of 1. Fix with a shared-memory tile. (10.5) 2 FLOP/12 B = 0.167 FLOP/B; at 1 TB/s this gives about 167 GFLOP/s $\ll$ 30.7 TFLOP/s: memory-bound. (10.6) To hide memory latency by switching warps, to fill the thousands of lanes, and to cover instruction dependencies and pipeline latencies.

Chapter 11. (11.1) A cached copy may be stale (status registers change on their own) and writes might never reach the device. (11.2) Each interrupt costs 4 μs, so 50% load $= 0.5/4\,\mu\text{s} = 125\,000$ interrupts per second. (11.3) $4\,194\,304$ B$/10^9$ B/s ≈ 4.2 ms; DMA leaves the CPU free to run other code during the transfer. (11.4) `svc` switches to supervisor mode, saves the return address and flags, jumps to the vector at `0x08`; the kernel reads the syscall number in r7, validates arguments, calls the driver, then returns to user mode. (11.5) At 4 GHz, 80 ns = 320 cycles: the data leaves the chip, crosses the memory controller and bus, and a DRAM row must be activated and sensed.